

Sparkling-Wasure en résumé

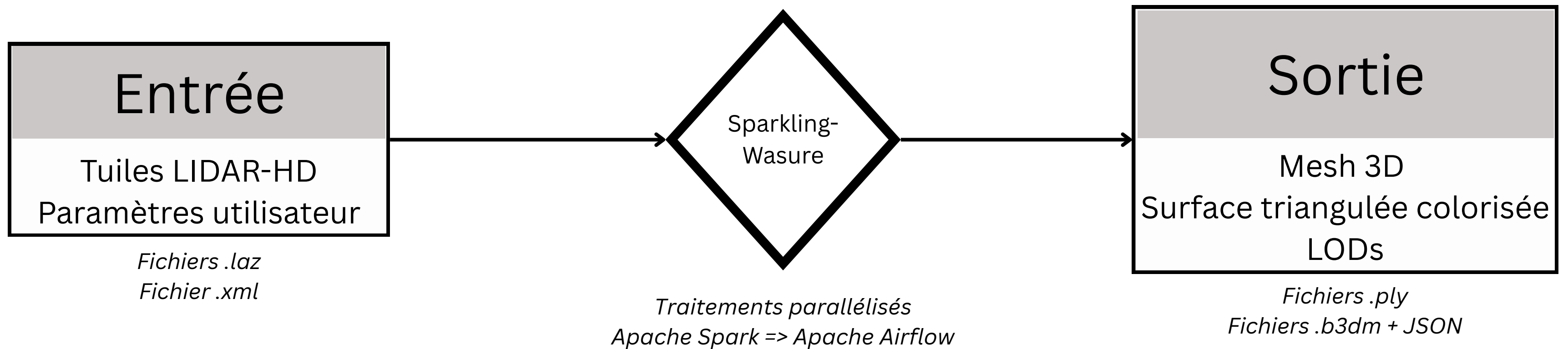
Sommaire

1. Glossaire	3
2. Le projet en gros	4 - 5
3. Wasure	6 - 38
i. Processus et Triangulation de Delaunay	7 - 25
ii. L'algorithme en entier	26 - 38
4. Colorize	39 - 41
5. 3DTiles	42 - 43
6. Docker	44 - 46
7. Références	47
8. Le code	48 - 63

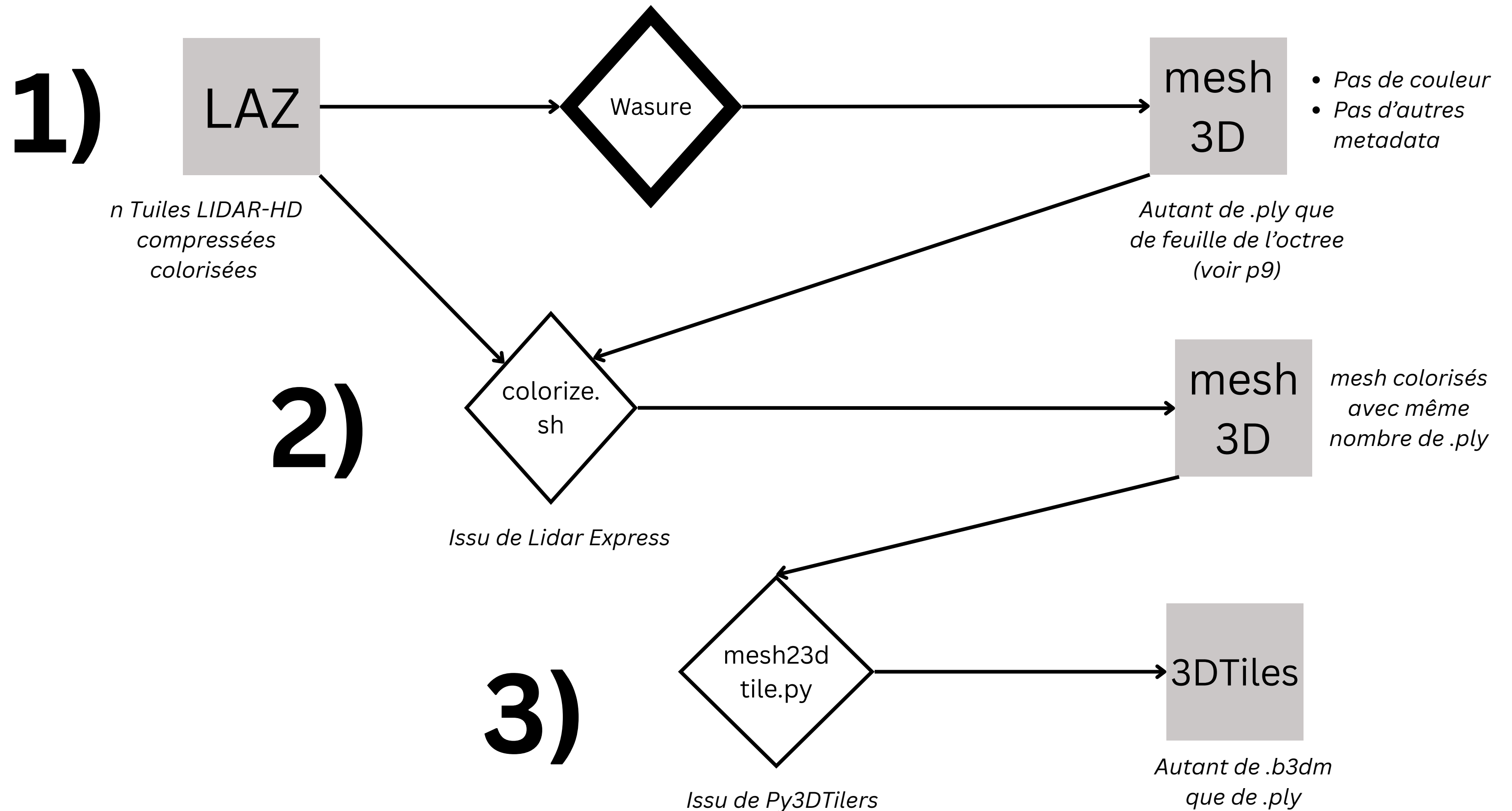
Glossaire

- 3DTiles : standard ouvert pour les ensembles de données géospatiales 3D
- .b3dm : fichier 3DTiles
- Cellule : triangle ou tétraèdre (mot template).
- DT : Triangulation de Delaunay
- .laz : fichier LIDAR-HD compressé
- .las : fichier LIDAR-HD décompressé
- Mesh 3D = Maillage 3D
- PCA (Principal Component Analysis) : méthode de description de la géométrie (diapo 25)
- PLY : données 3D (nuage de point, maillage, etc.)
- .ply : fichier contenant de la données 3D

Le projet en gros

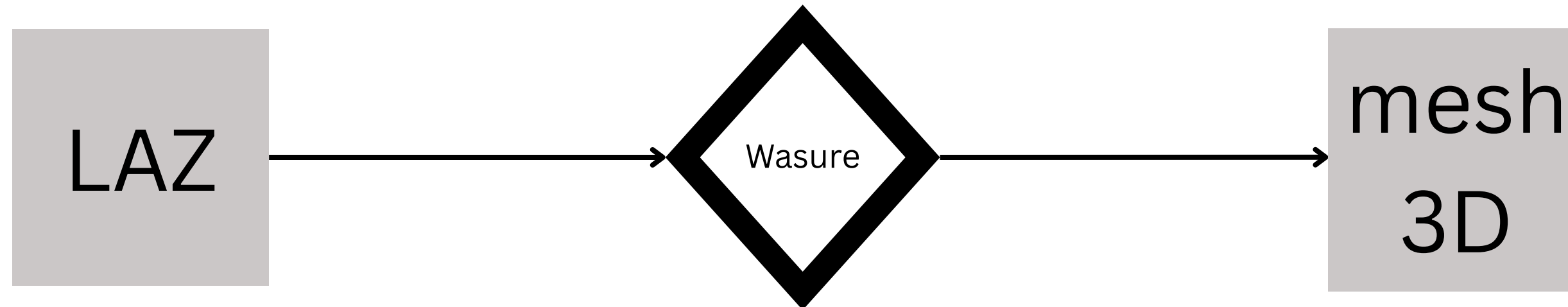


Le projet en gros



Wasure

1)



DT = Triangulation de Delaunay

L'algorithme

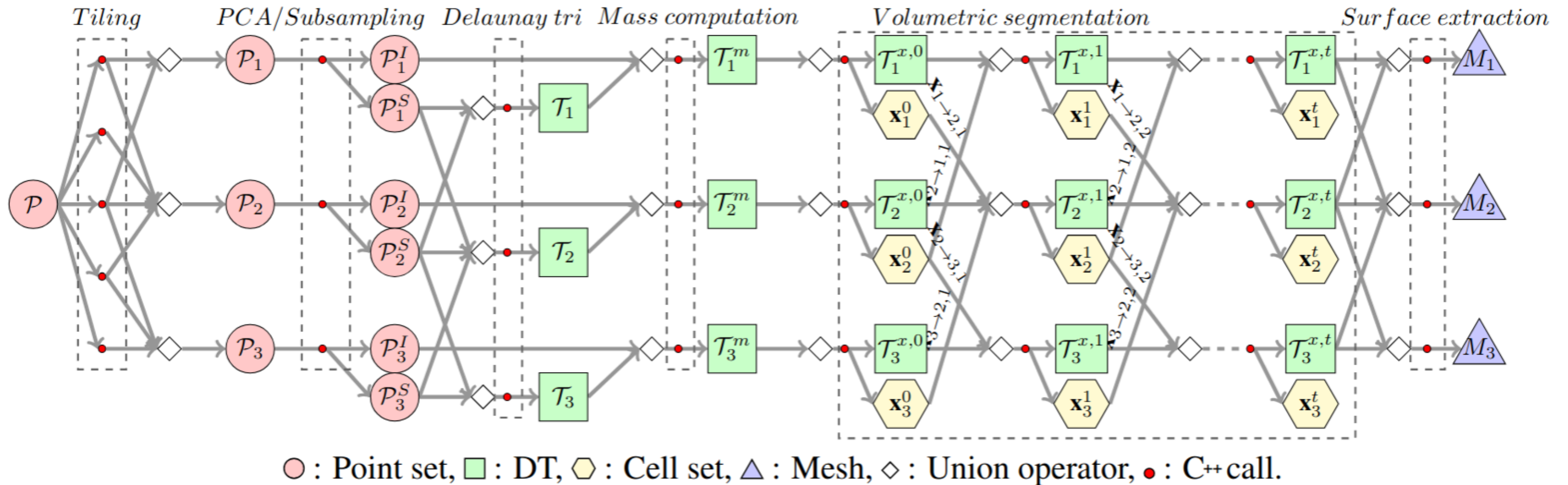


Schéma de l'algorithme de reconstruction de surface à partir d'un nuage de point LIDAR-HD (ref 5)

LAZ

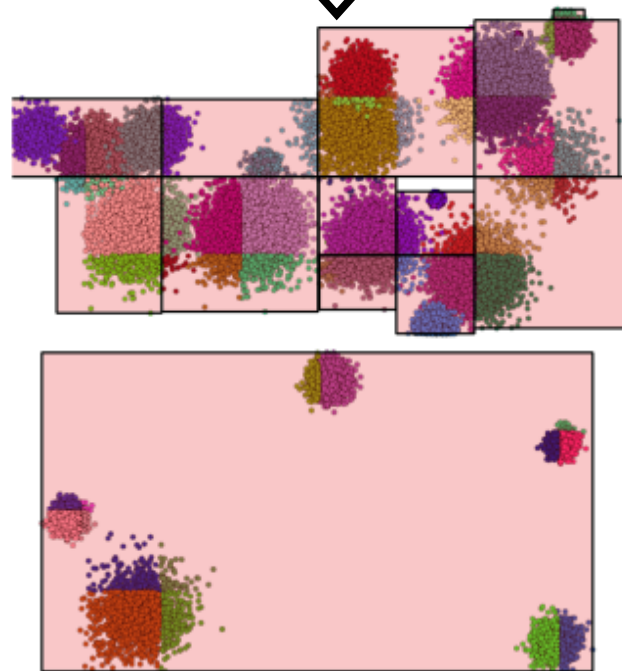
Octree

Réduction du
nombre de
points

DT
locale

Algo
de DT

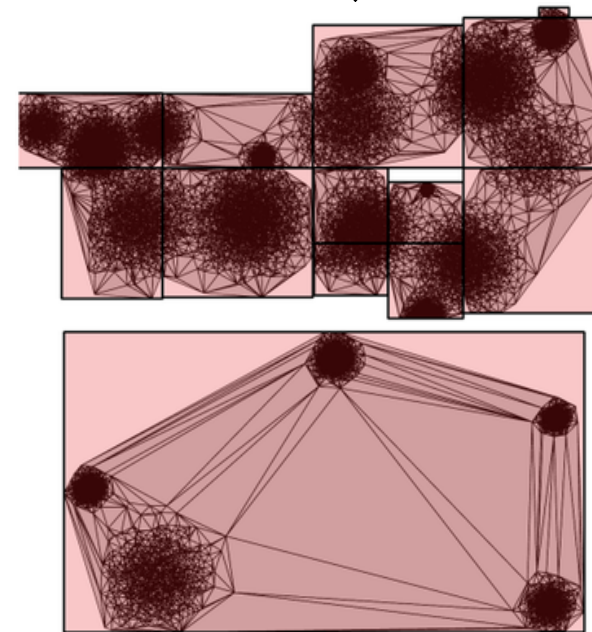
*n Tuiles
LIDAR-HD*



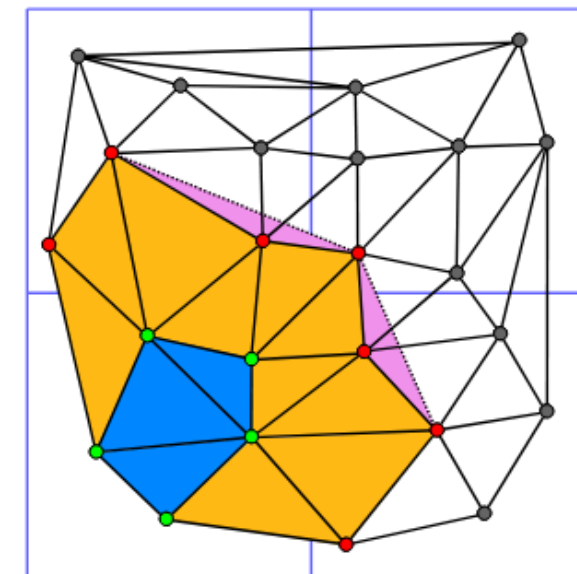
*Segmentation par tuile =>
Segmentation par nbr de points*

- Voisinage de chaque point
- PCA (Principal Components Analysis)

=> Nouveau jeu de données moins lourd
sans perte d'information géométrique



Triangulation de Delaunay locale

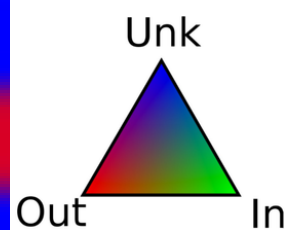
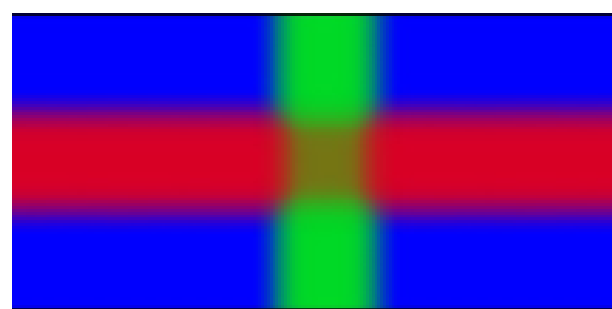
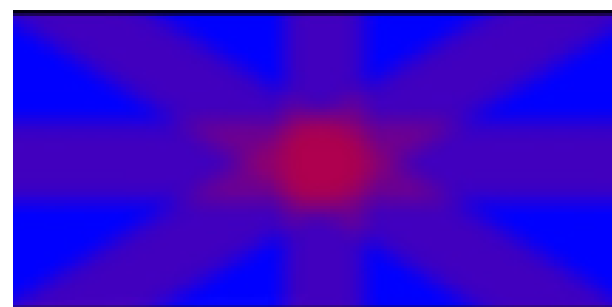


Résultats dans une tuile

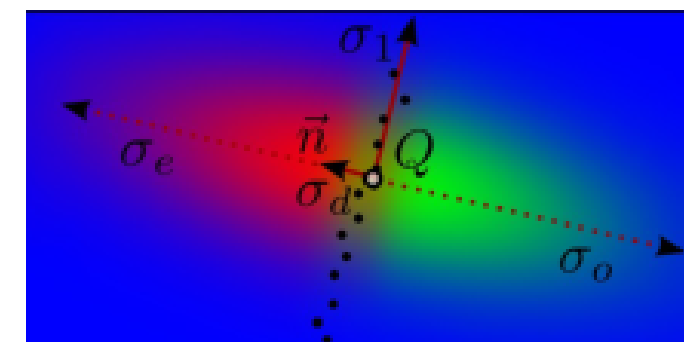
Segmentation
volumétrique

Fonction
de masse

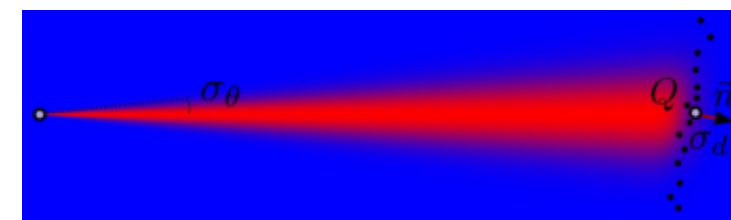
Extraction de la
surface



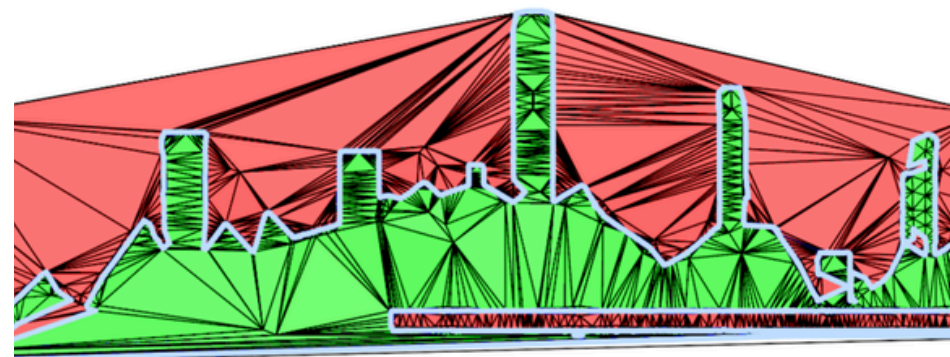
*Zone autour des points pas uniquement
"Vide" ou "Plein" ou "Incertain"*



Par rapport à la normal au point



Par rapport au centre du capteur



*Surface = Les triangles entre
tétraèdres labellisés "empty" et
"occupied"*

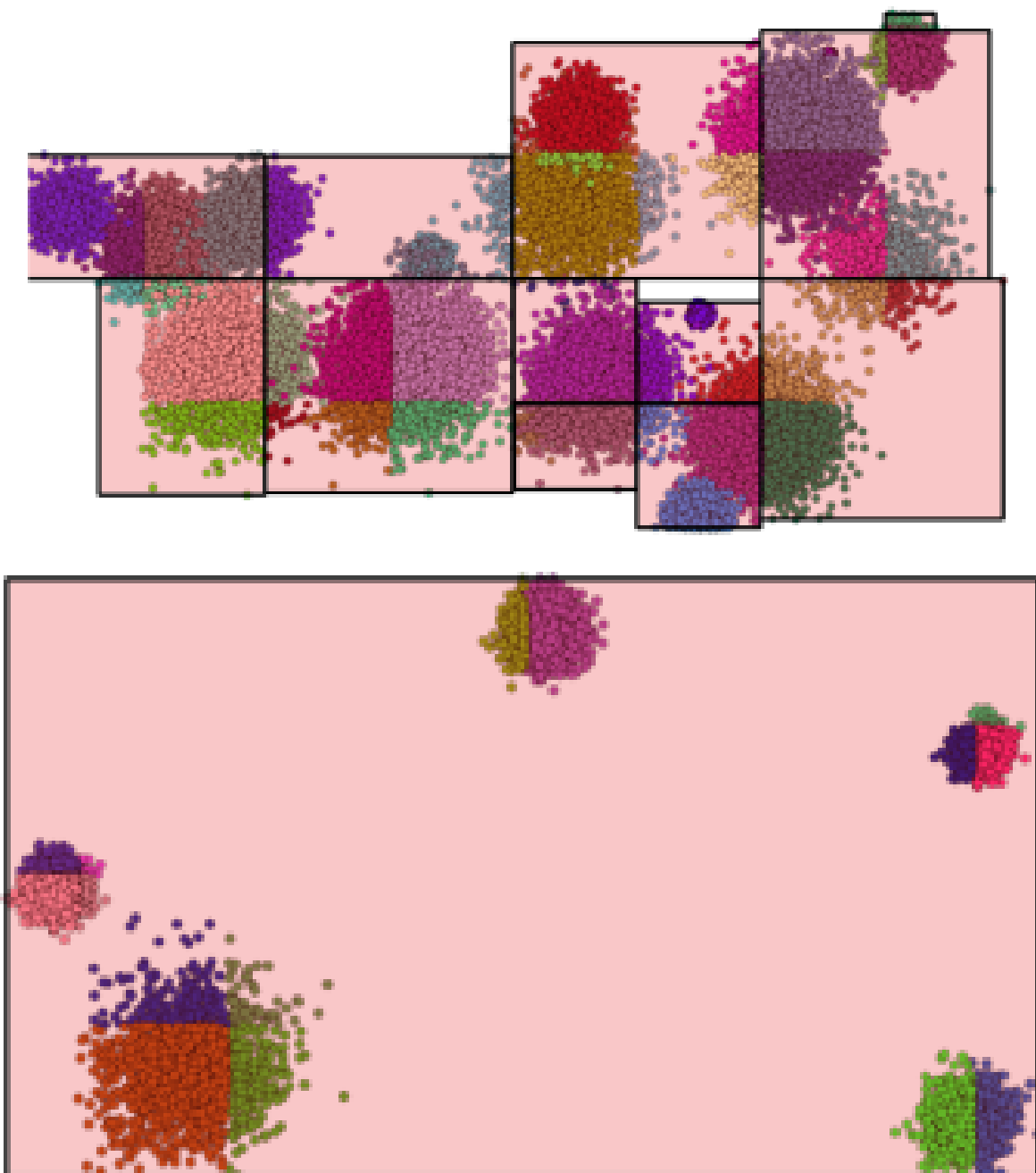
mesh
3D

*Autant de
.ply que de ?*

LAZ

Octree

*n Tuiles
LIDAR-HD
compressées*



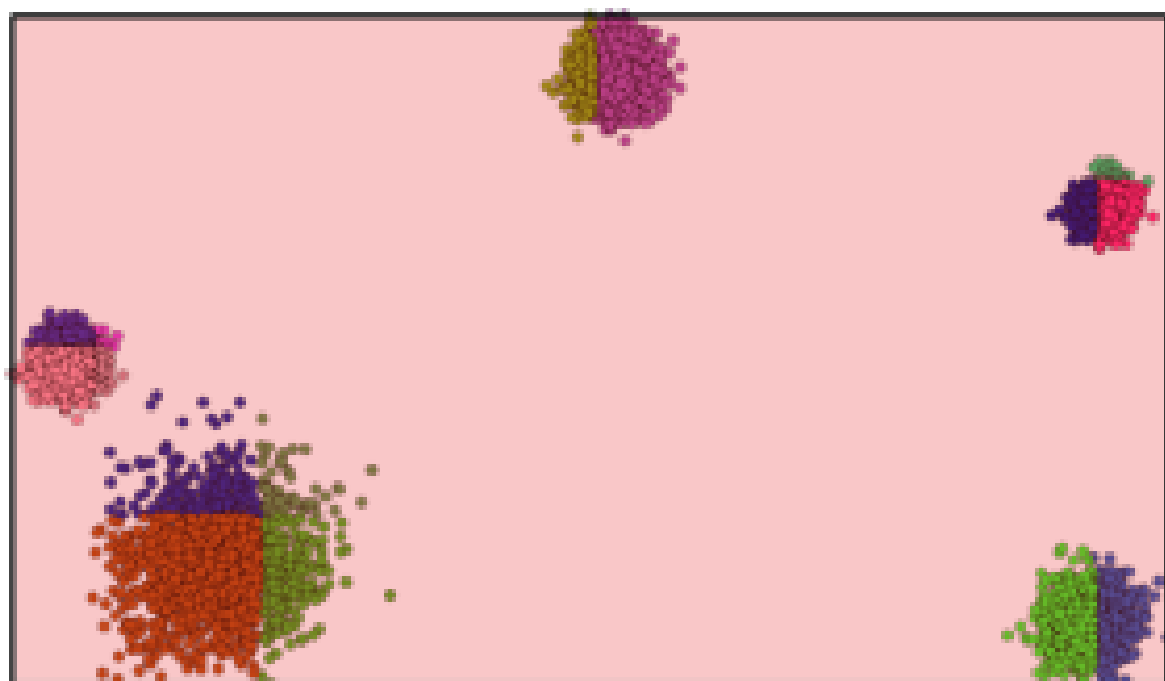
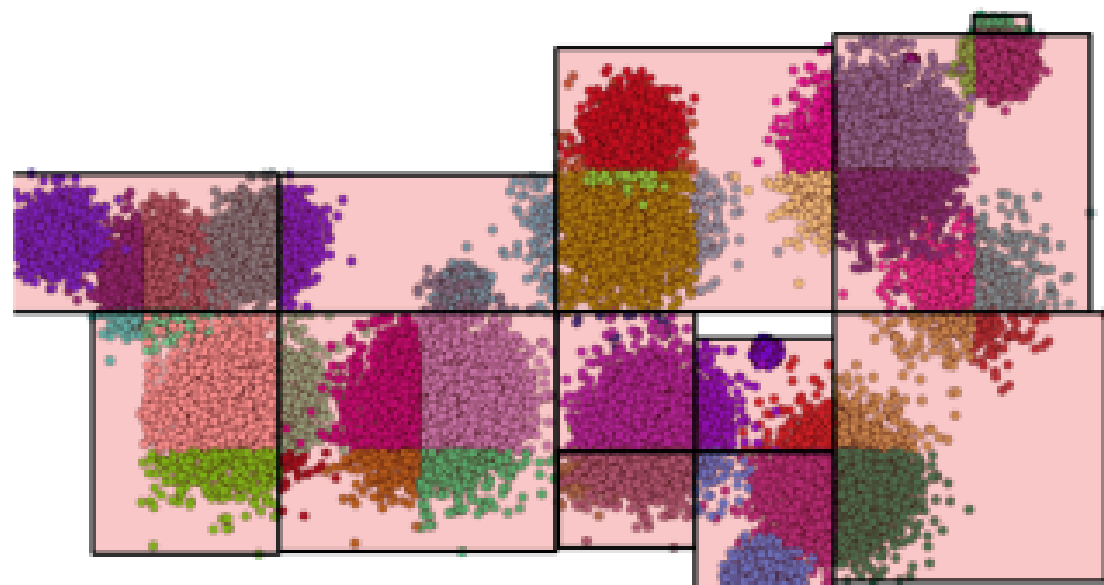
Segmentation Octree (ref 3)

- Une tuile LIDAR-HD de 1km² contient trop de point à traiter.
- Les tuiles LIDAR-HD en entrée sont d'abord sous-segmentées en sous-tuiles.
- Ces sous-tuiles permettent de créer un Octree, avec comme critère de répartition le nombre de point contenu dans chaque nœud.
- La méthode est “Bottom-Up”, on fusionne les sous-tuiles si le nombre de point suivant la fusion est inférieur au nombre de point souhaité dans une feuille de l’octree.

LAZ

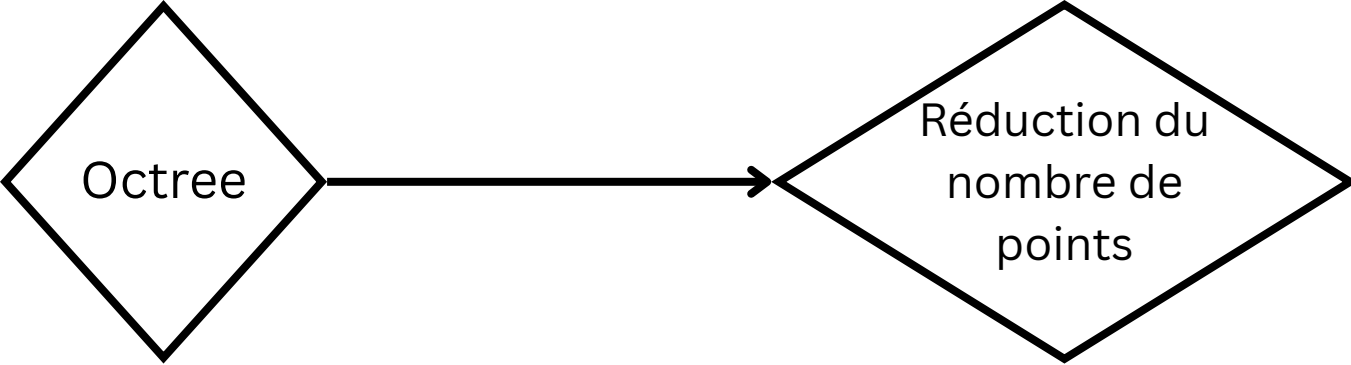
Octree

*n Tuiles
LIDAR-HD
compressées*



Segmentation Octree (ref 3)

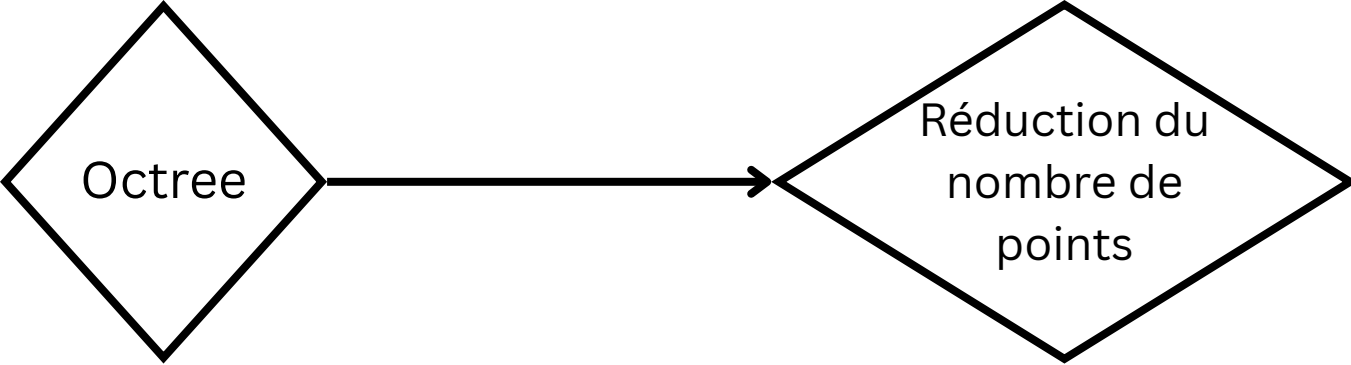
- Si une sous-tuile possède déjà trop de point, alors elle reste comme cela.
- Cette méthode permet de ne jamais avoir toutes les tuiles LIDAR-HD à un seul et même endroit de stockage.
- Chaque critère est un paramètre utilisateur.
- Chaque feuille de l'Octree est distribué sur un RDD Spark, avec un id. On parlera par la suite de tuile.



PCA

PCA (Principal Components Analysis) : méthode de description de la géométrie

- Calcul d'un PCA pour chaque point.
- Estimation d'un repère local, description de la dispersion locale et calcul de la normale locale.
- La normale est orientée à l'aide de la position du capteur.



Subsampling

Subsampling : Créer un nuage simplifié par sous-échantillonnage adaptatif

- Sélection aléatoire de points candidats.
- Ajout d'un point seulement s'il apporte une information géométrique nouvelle.
- Le critère est basé sur la distance anisotrope (définition après) dans le repère PCA local.

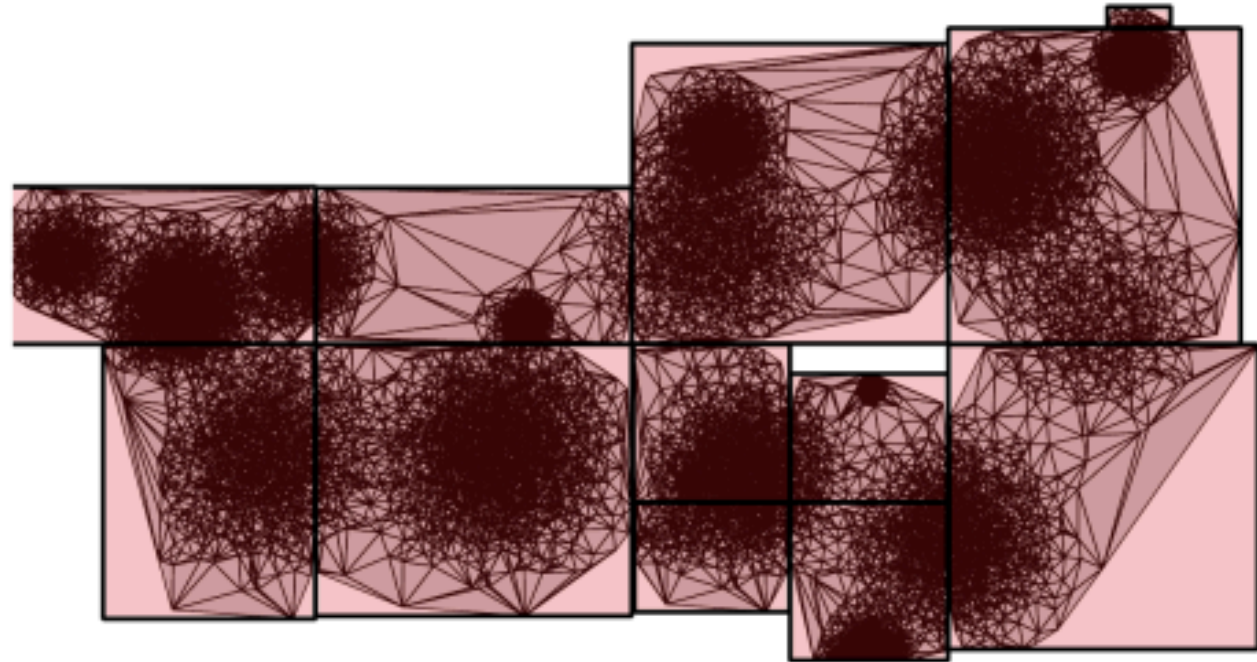
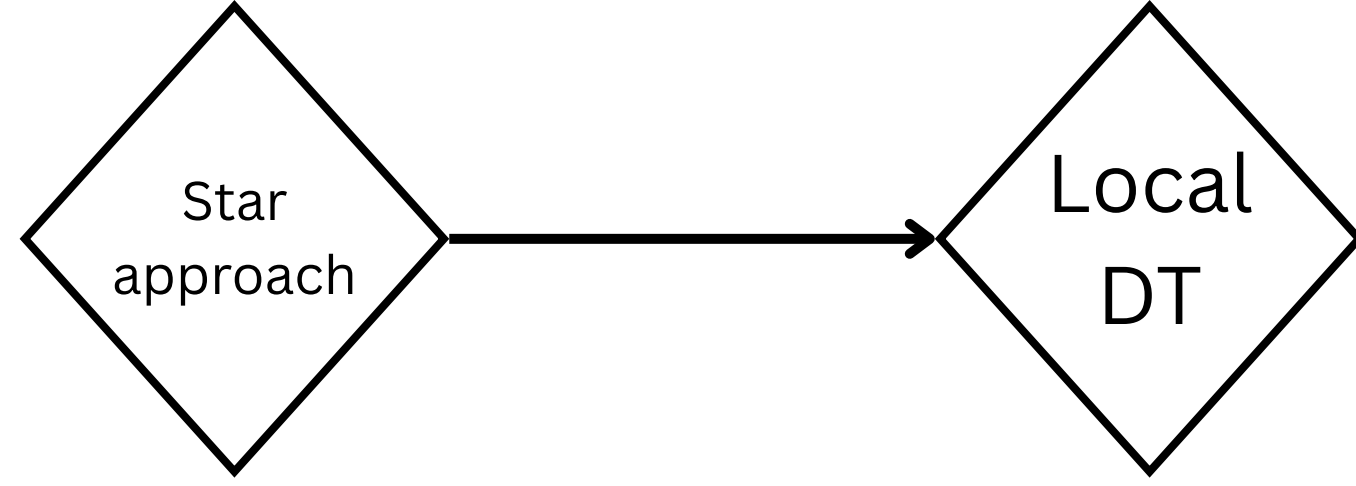
=> On obtient une forte réduction sur les surfaces planes et la conservation des zones complexes.

On obtient un nouveau pointset plus léger pour le traitement.

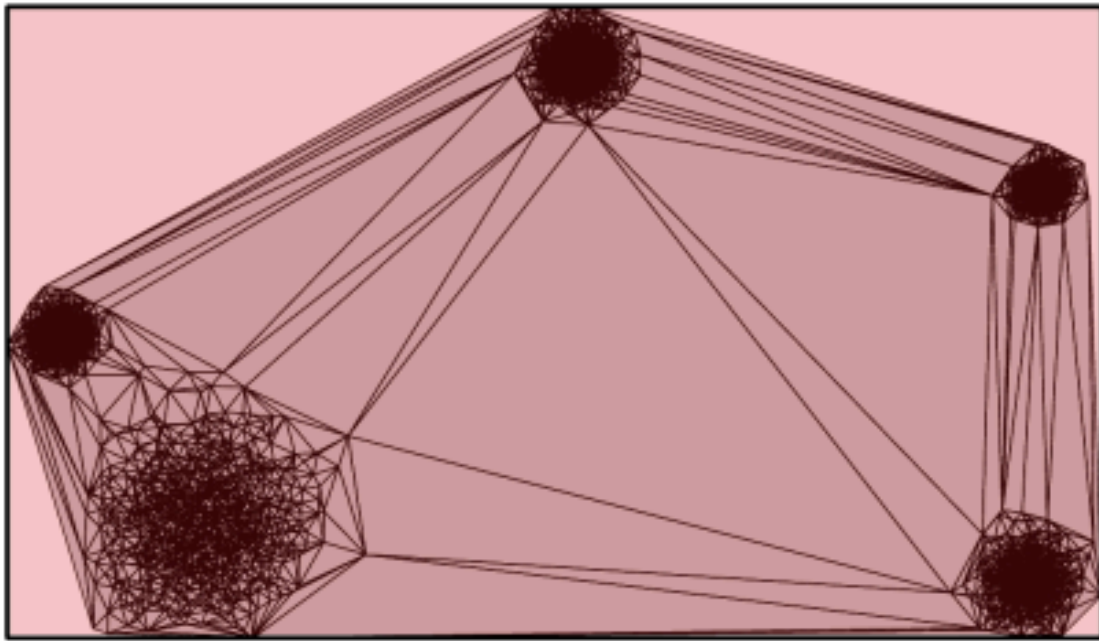
Distance anisotrope

- Différente de la distance euclidienne car la mesure n'est pas uniforme
- La distance entre 2 points est évaluée différemment en fonction de leur appartenance à une même surface ou non.
- Les directions où le nuage de point s'étale beaucoup tolèrent de plus grands écarts, tandis que la direction normale à la surface impose un seuil beaucoup plus strict.

=> Ainsi, deux points peuvent être considérés “proches” (distance anisotrope) s'ils sont éloignés (distance euclidienne) dans le plan, mais “éloignés” s'ils s'écartent en hauteur (par rapport au plan) même faiblement.

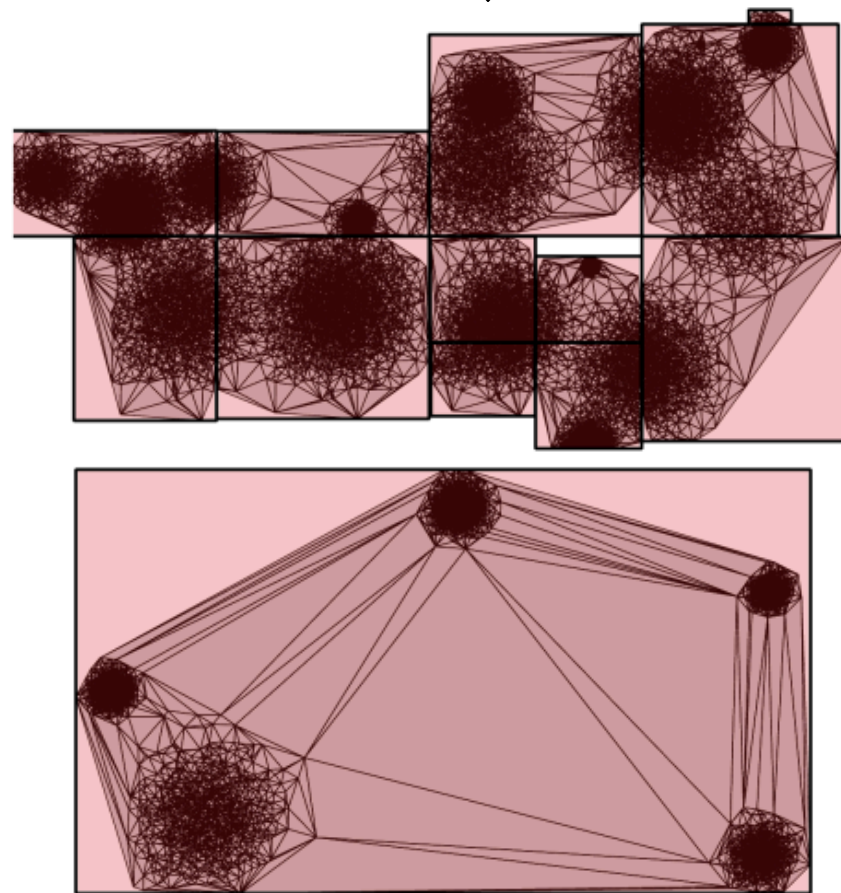
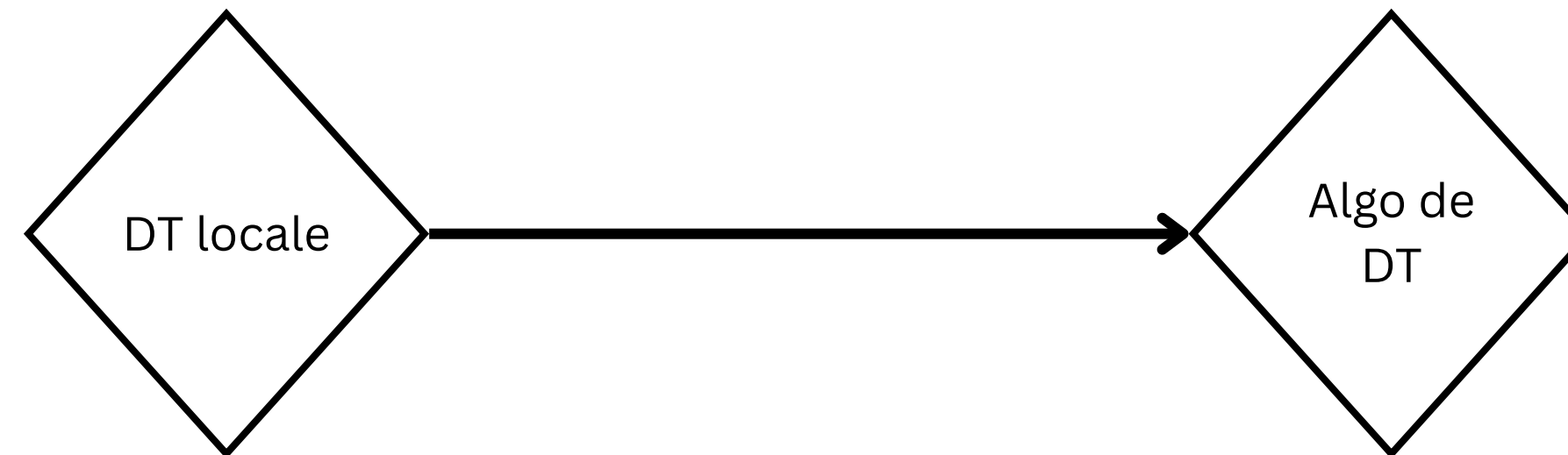


- **Pour chaque tuile, la triangulation de Delaunay est appliquée localement.**

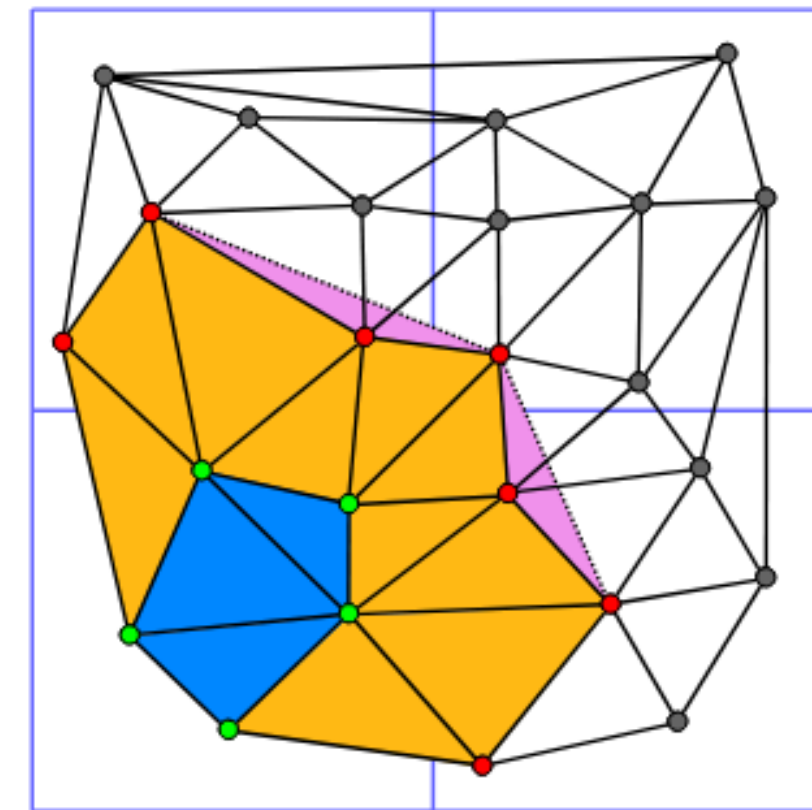


Triangulation de Delaunay locale (ref 3)

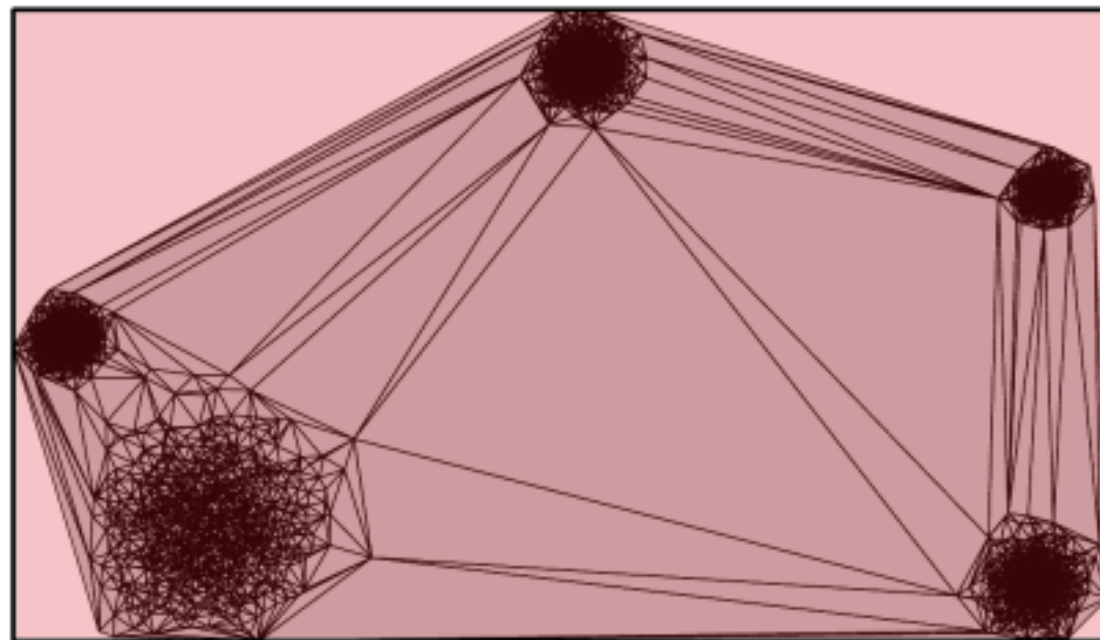
L'algorithme de Triangulation de Delaunay distribué



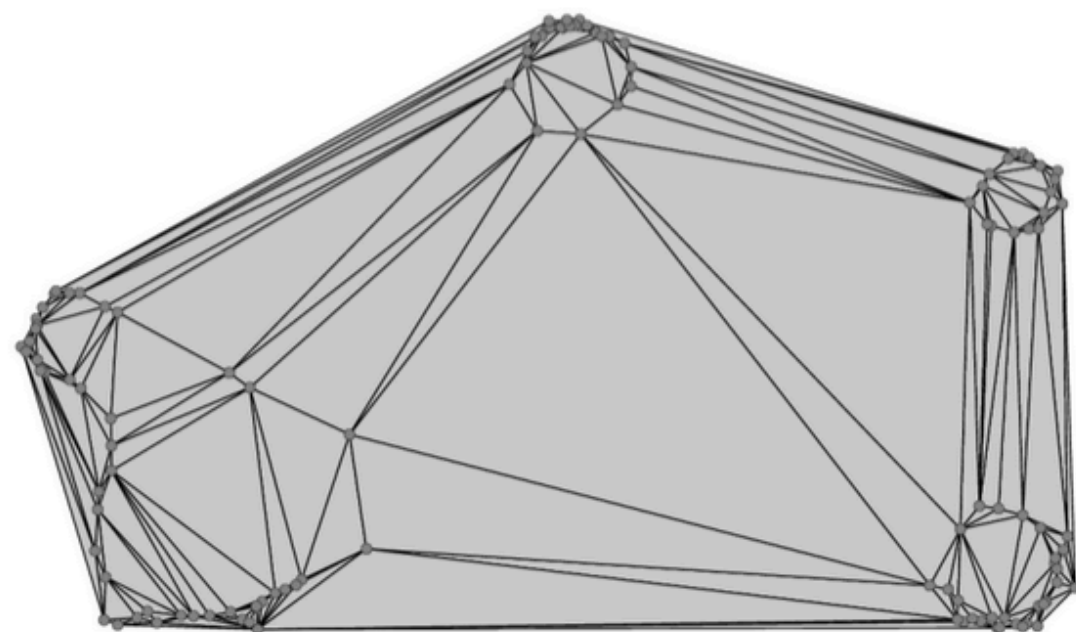
Triangulation de Delaunay locale



Résultats dans une tuile



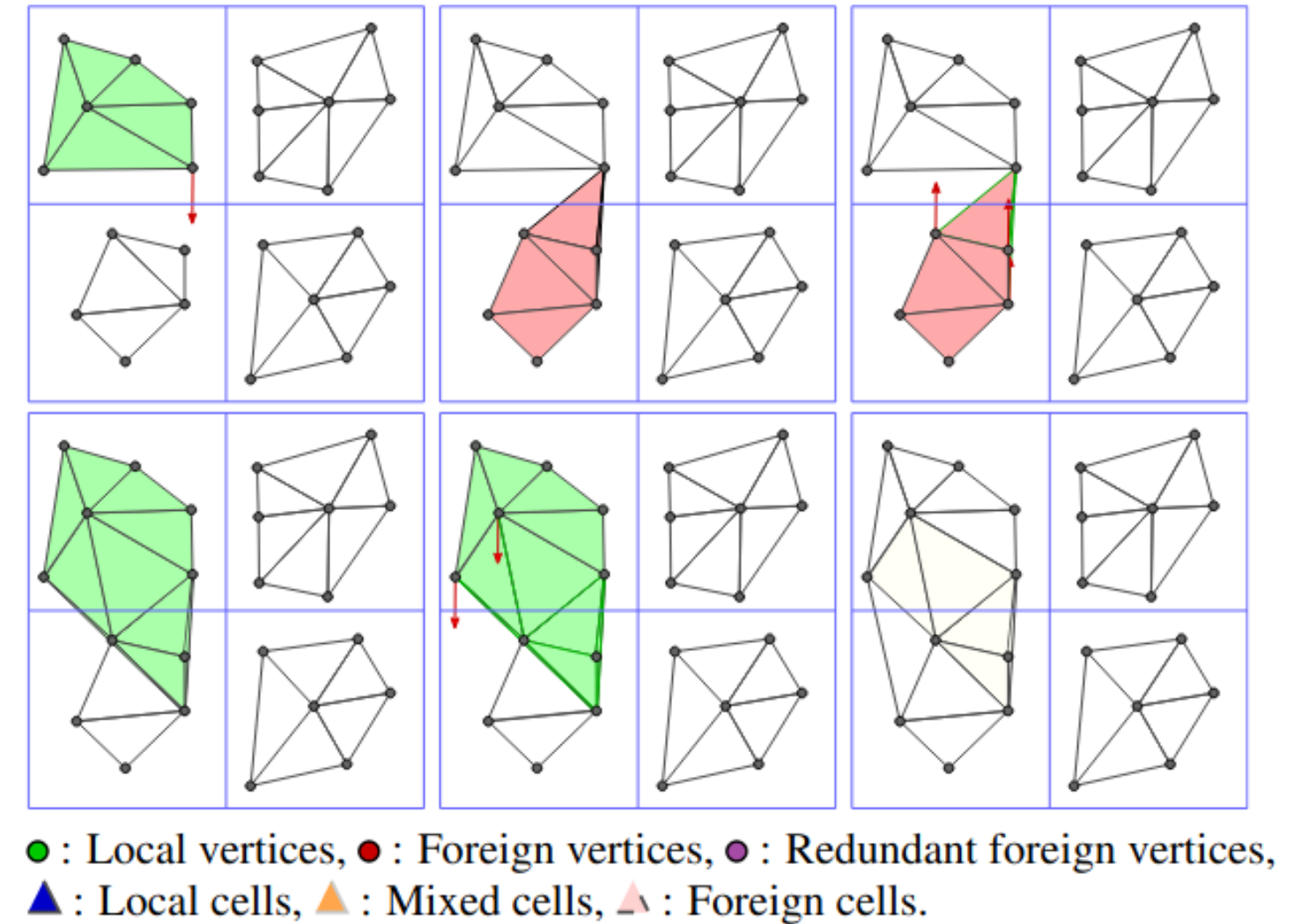
Triangulation de Delaunay locale (ref 3)



Triangulation de Delaunay locale avec uniquement les "cellules fragiles" (ref 6)

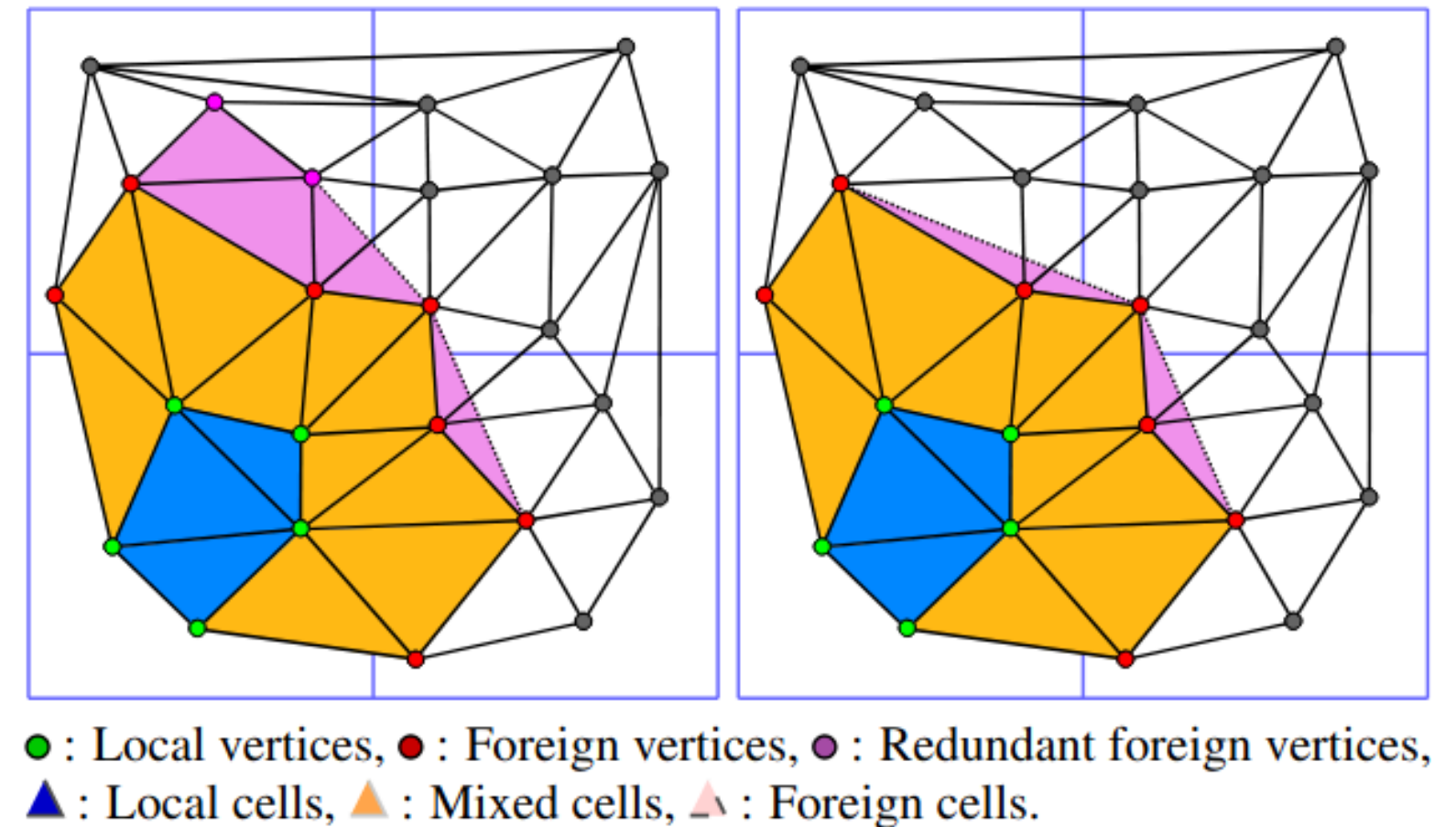
- On sélectionne dans la DT les cellules dont l'enveloppe convexe touche les bords de la tuile. Ils ont une forte chance d'être modifié par les autres tuiles.
- On peut voir les autres cellules comme "trop compact" pour voir leur lien "se briser" par l'ajout de points appartenant à d'autres tuiles.
- On obtient donc des DT locales plus légère pour le traitement.

- Dans chaque tuile il y a donc une DT locale simplifiée.
- La triangulation se fait par échange d'information sur le voisinage des points entre les DTs. L'objectif est que chaque point possède localement les même informations sur son voisinage que dans la DT globale.
- Quelques points sont envoyés et intégrés/triangulés aux DTs locales adjacentes, qui elles même renvoient les points qui devrait être adjacent à ce point afin d'être intégré à la DT émettrice et ainsi de suite.
- Le message envoyé à une tuile adjacente contient quelques points, l'id de la tuile émettrice et l'id de la tuile réceptrice.
- Chaque tuile renvoie les voisins du points s'ils n'ont pas déjà été envoyés à la tuile émettrice. L'algorithme s'arrête lorsqu'il n'y a plus de point à renvoyer.
- Envoyer quelques points aléatoires au départ suffis à l'algorithme pour finir sur une solution juste.



*Illustrations de l'algorithme de
Triangulation de Delaunay (ref 4)*

- À la fin, une tuile possède les cellules de sa DT locale (bleu), les cellules qu'elle partage avec ses points voisins (jaune), et des cellules non présents dans la DT globale (rose) mais présent dans les DTs locales pour assurer que l'enveloppe convexe de tous les points est triangulée.
- Ces derniers triangles sont formés à l'aide de points étrangers (ni locaux, ni voisins). Ils sont supprimés par simplification, sans modifier le voisinage de Delaunay des points locaux.
- L'algorithme de triangulation de Delaunay distribué est obligé de créer des cellules qui n'existent pas dans la DT globale afin de rendre chaque DT locale convexe. Sinon les fonctions de CGAL ne pourraient pas fonctionner pour la suite.



*Illustrations de l'algorithme de
Triangulation de Delaunay (ref 4)*

L'algorithme de Triangulation de Delaunay distribué

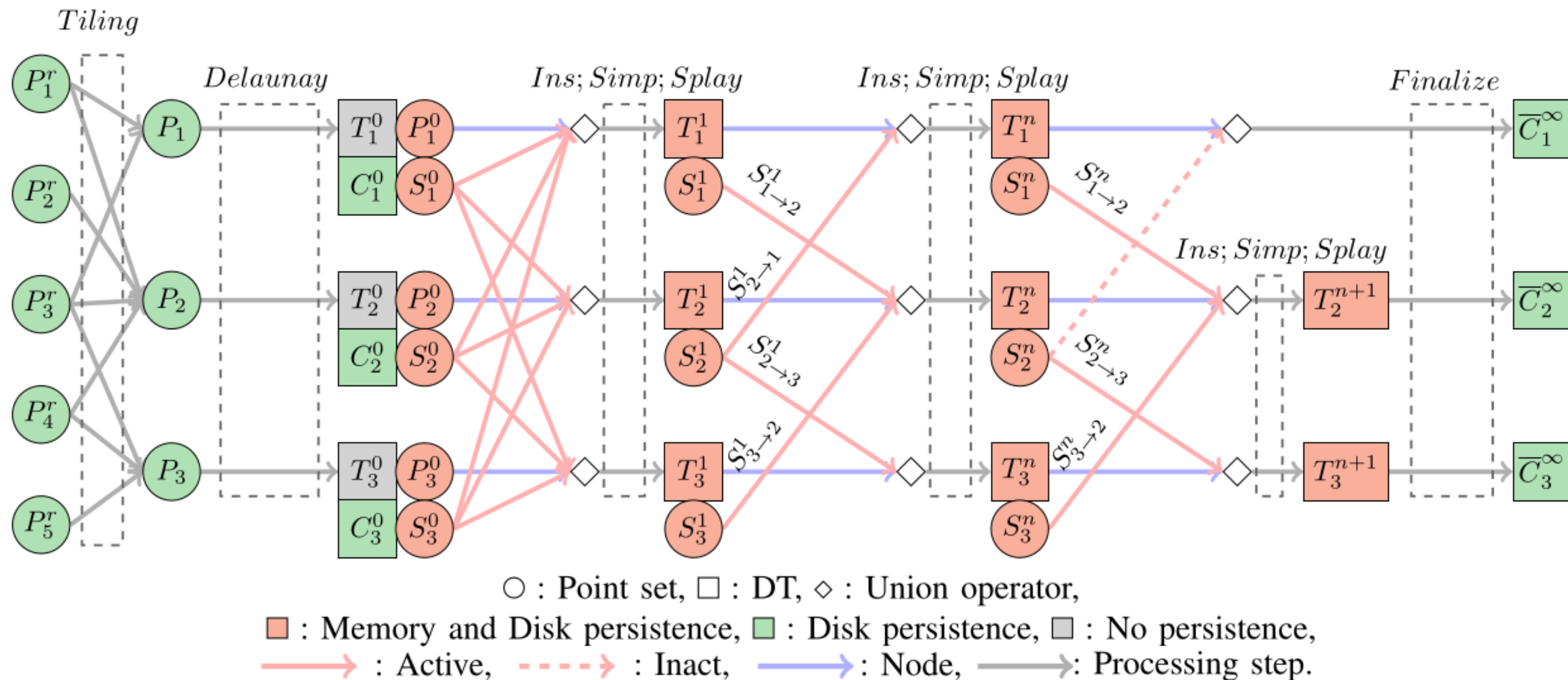


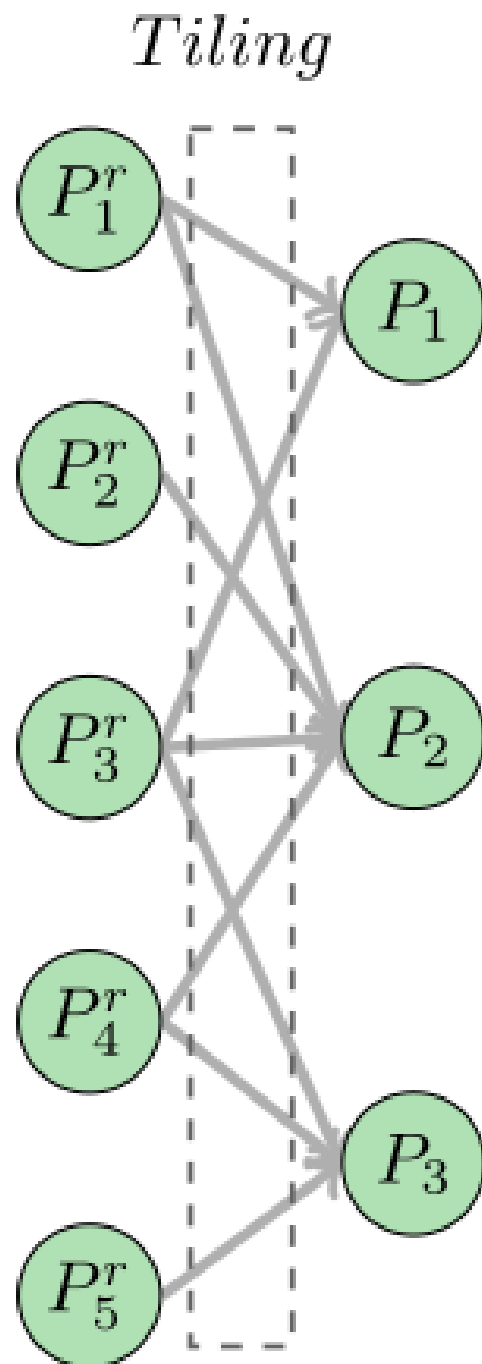
Schéma de l'algorithme de triangulation de Delaunay distribué (ref 3)

L'étape PCA/Subsampling n'est pas représentée sur ce schéma.

En simplifié

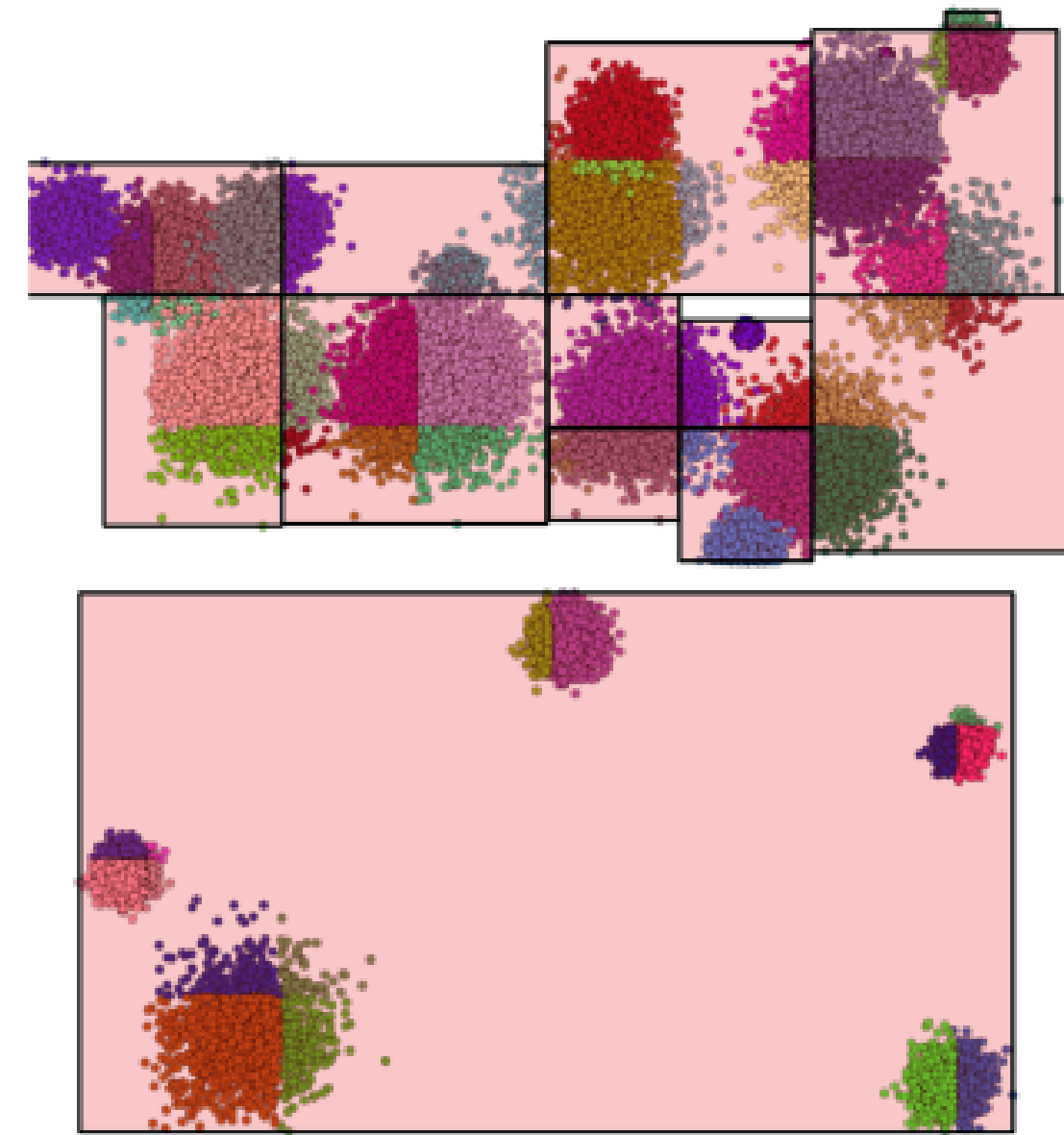
Octree

Tuiles
LIDAR-HD



P : Point set
d'une tuile

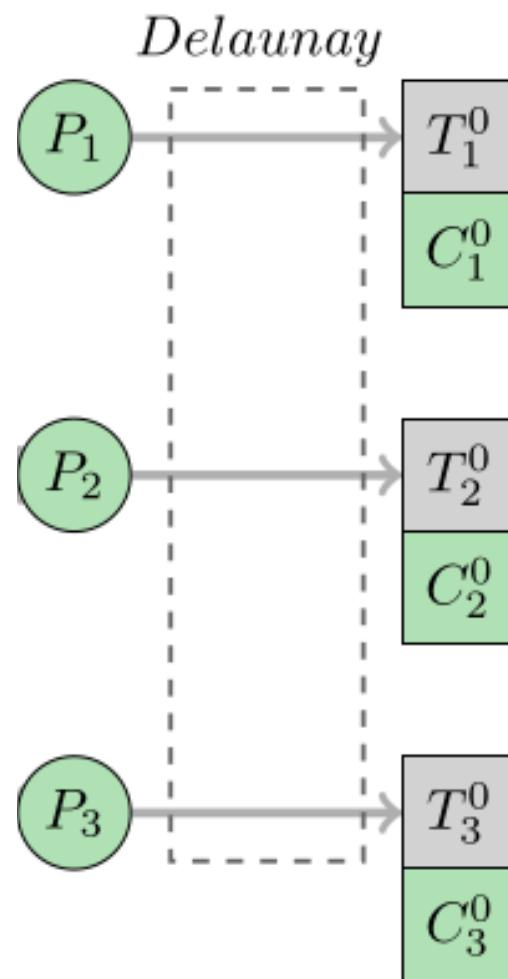
=



*Segmentation par tuile =>
Segmentation par nbr de points*

En simplifié

Initial Local DT

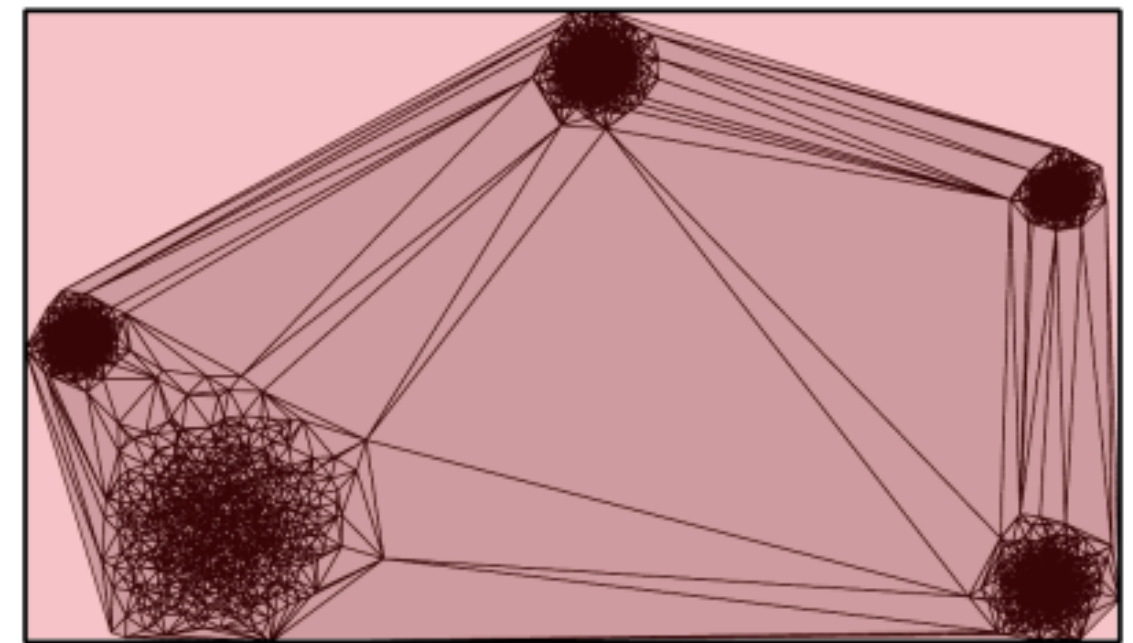


**T^0 : Triangulation
de Delaunay locale
simplifiée**

**C^0 : Cellule “solide”, le
complémentaire de T^0**

=

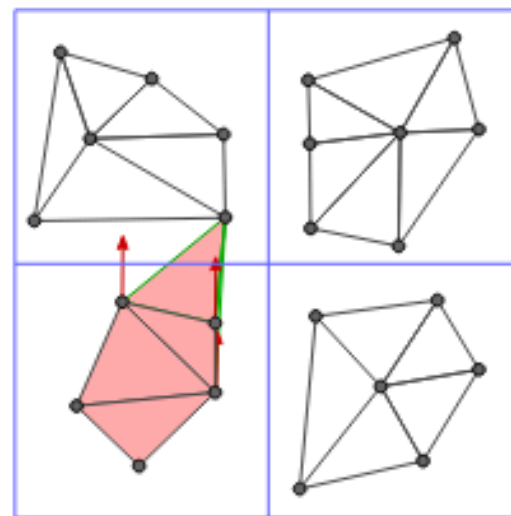
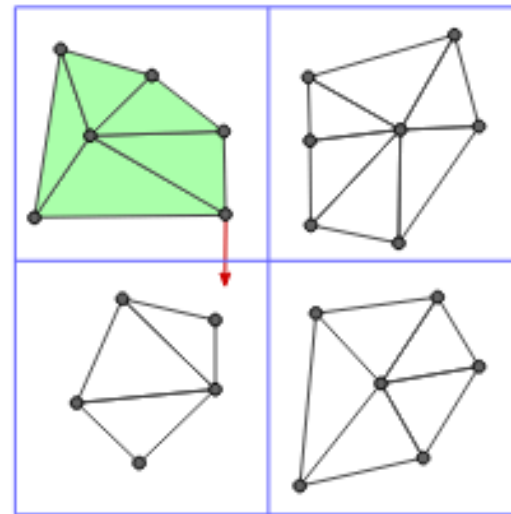
T^0 : Triangulation de Delaunay locale simplifiée



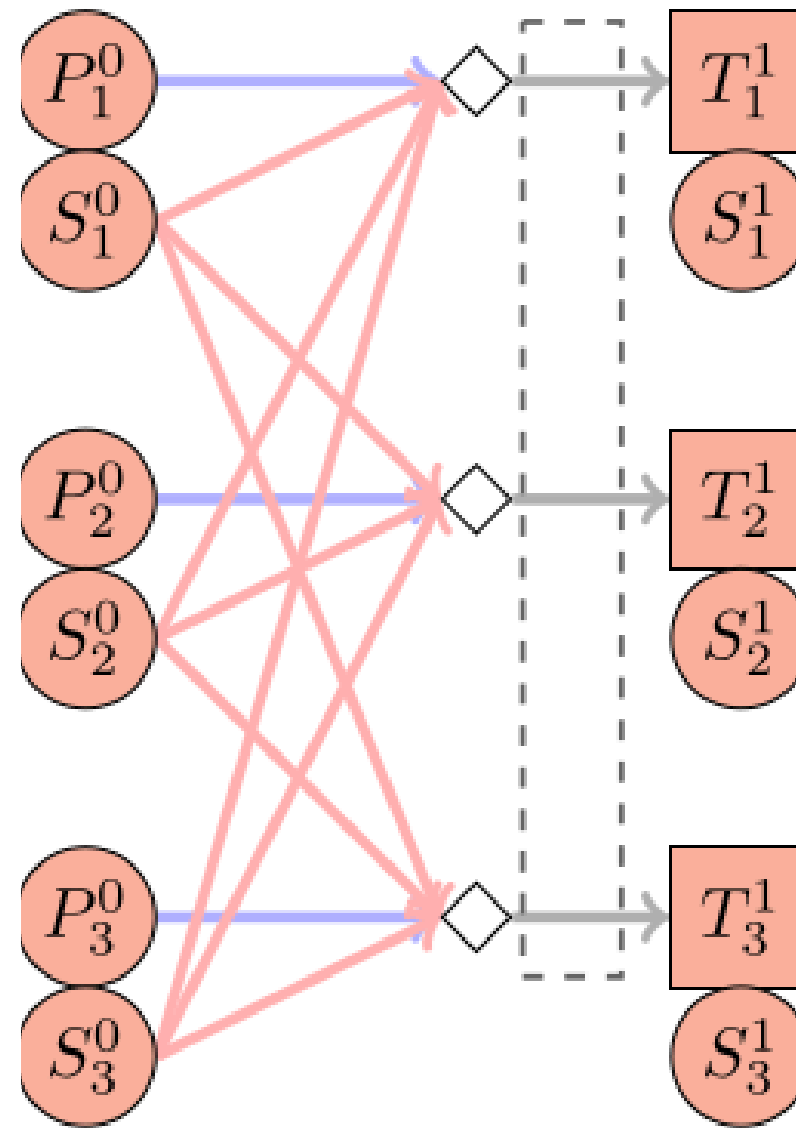
$T^0 + C^0$

En simplifié

P^0 : Les points a intégré de départ, issues de T^0



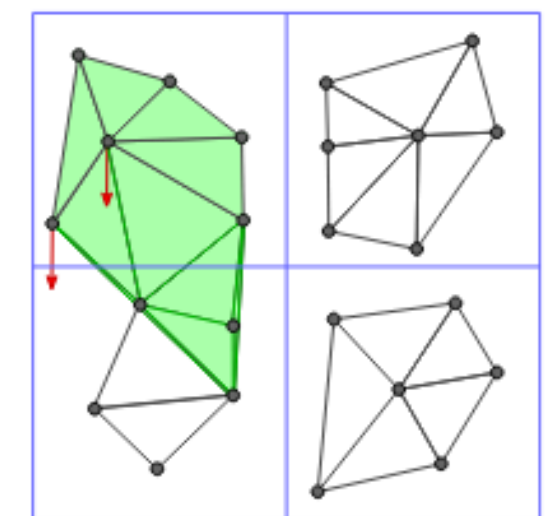
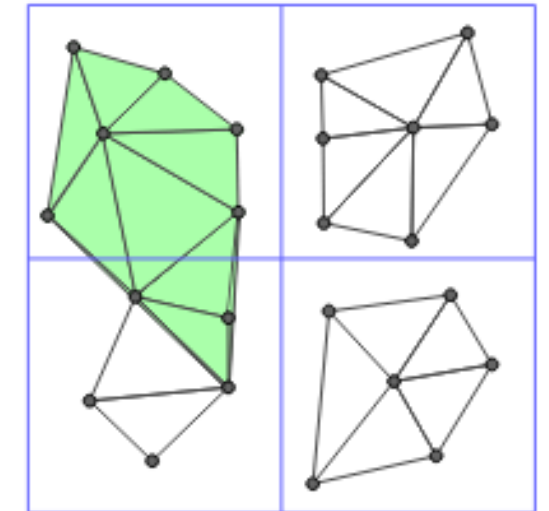
S : Les points voisins



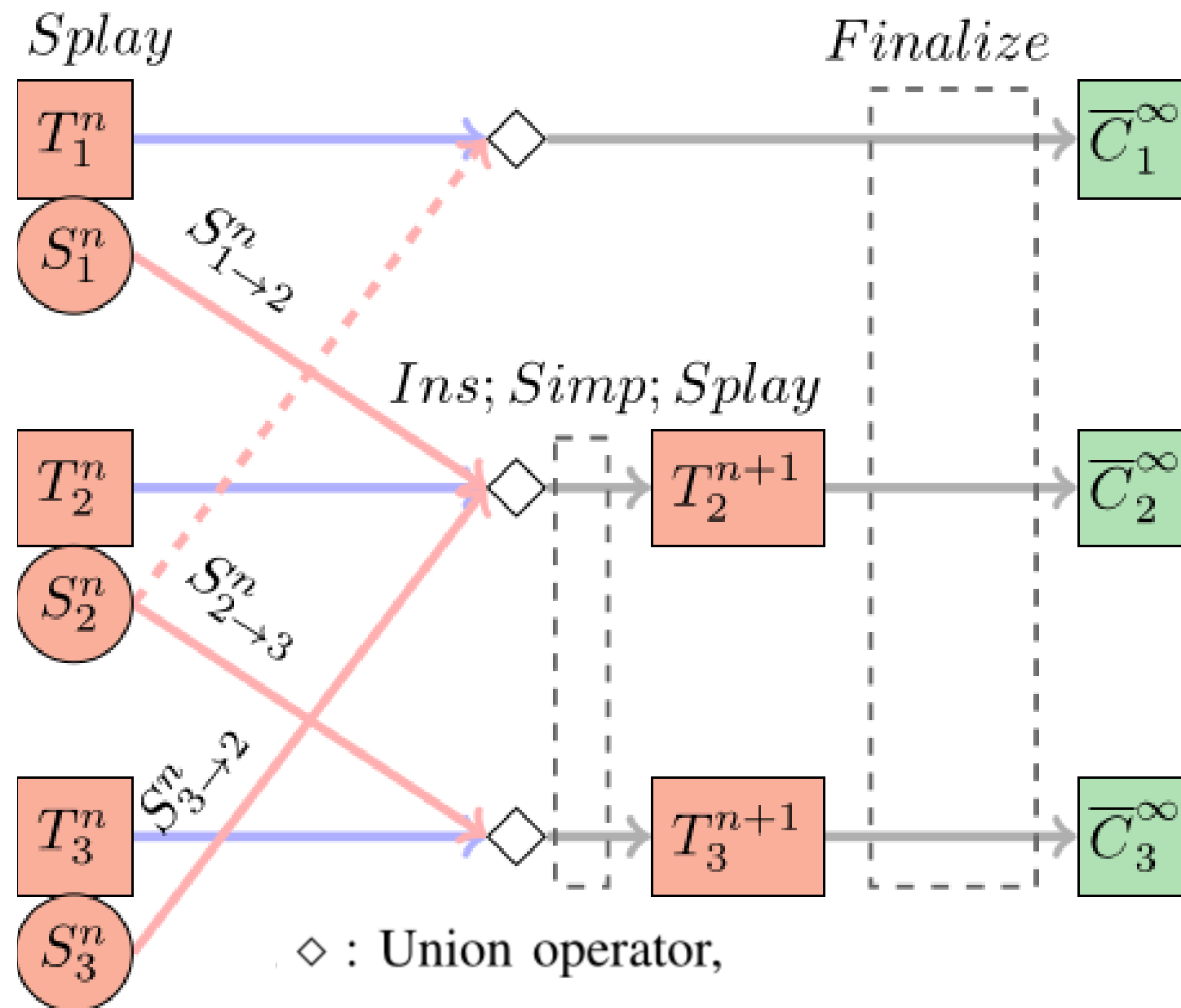
$\diamond$: Union operator,

T : Triangulation de Delaunay locale

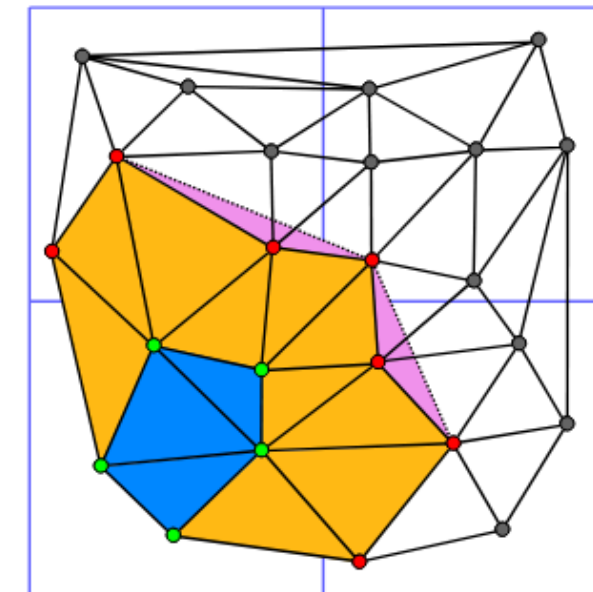
S : Les points voisins



En simplifié

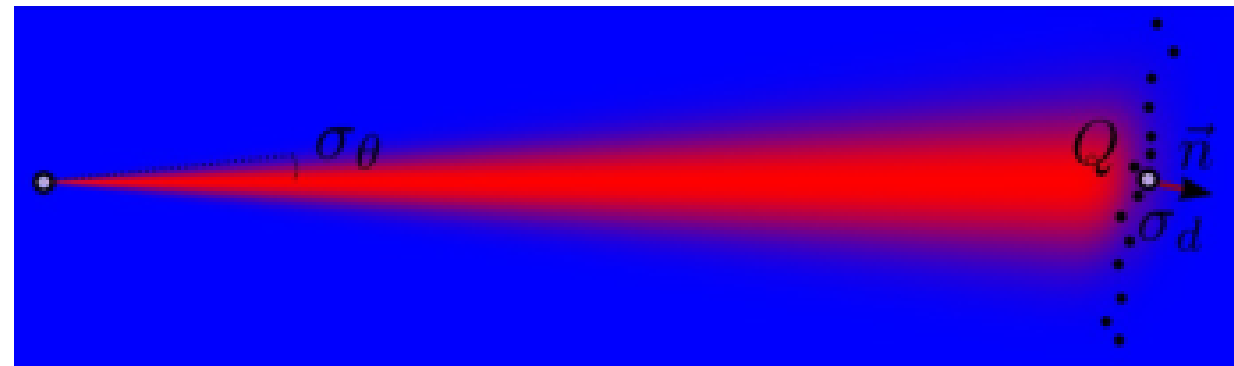
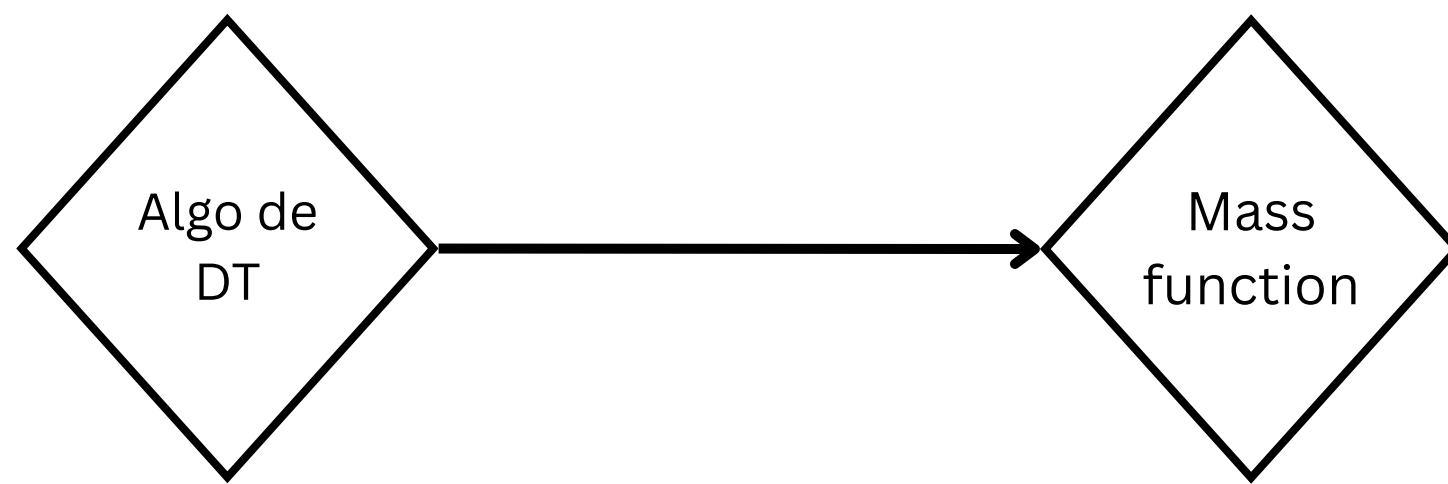


C_{barre} : le complémentaire de C

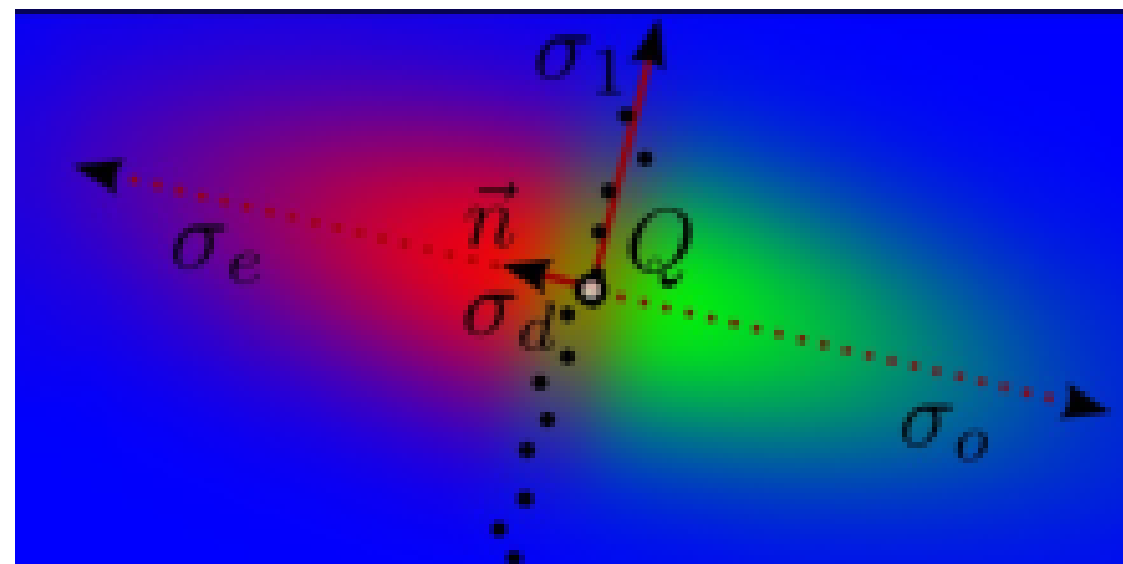


$C \text{ union } C_{\text{barre}} = \text{DT globale}$

Certaines tuiles vont
converger avant d'autres



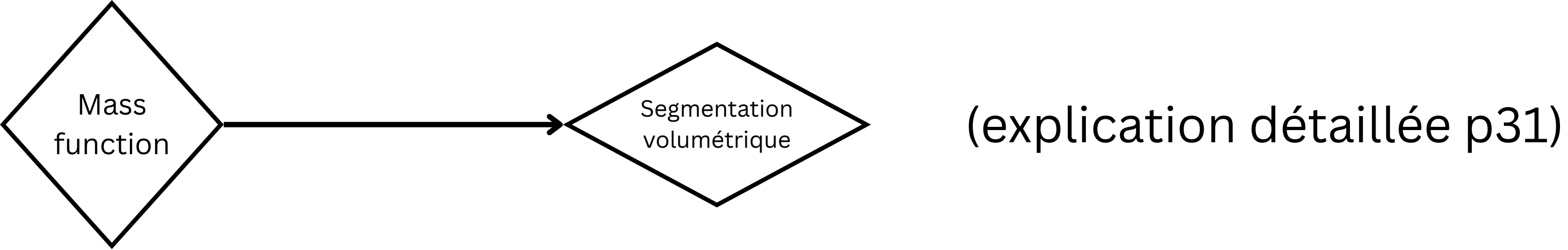
Par rapport au centre du capteur



Par rapport à la normale au point

Illustration de la fonction de masse (ref 1)

- **La fonction de masse est calculée pour chaque point. Elle permet de déterminer l'extérieur et l'intérieur autour du point.**
- **La normale au point est calculée à l'aide d'une PCA (voir p27). Elle est orienté vers l'extérieur grâce à la position du capteur.**
- **Il faut que la position du capteur soit connu pour définir l'extérieur de l'intérieur.**



En fusionnant chaque calcul de fonction de masse, chaque alentour de point possède un potentiel de labellisation, caractérisé dans un spectre entre “Vide”, “Plein” et “Incertain” (comme “Potentiellement Plein” par exemple).

Un score moyen de la fonction de masse est attribué au barycentre des tétraèdres.

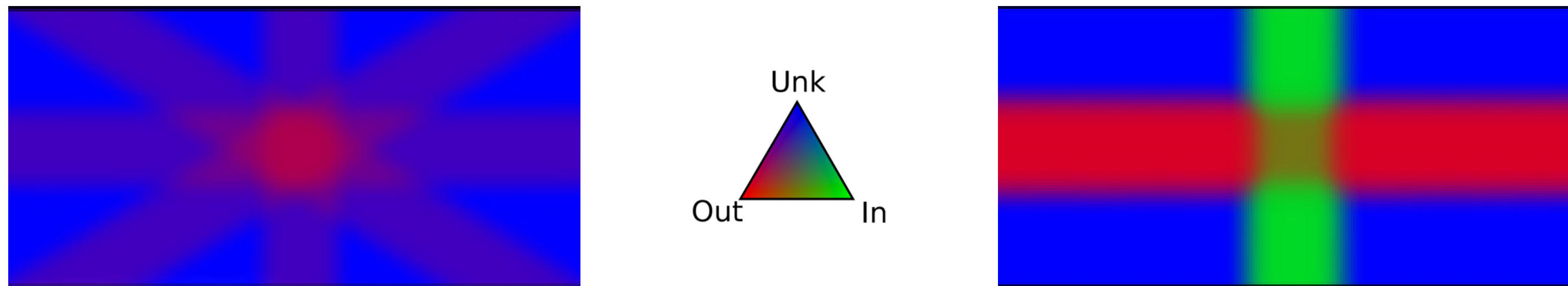


Illustration de fusion de fonction de masse (ref 1)

L'algorithme en entier

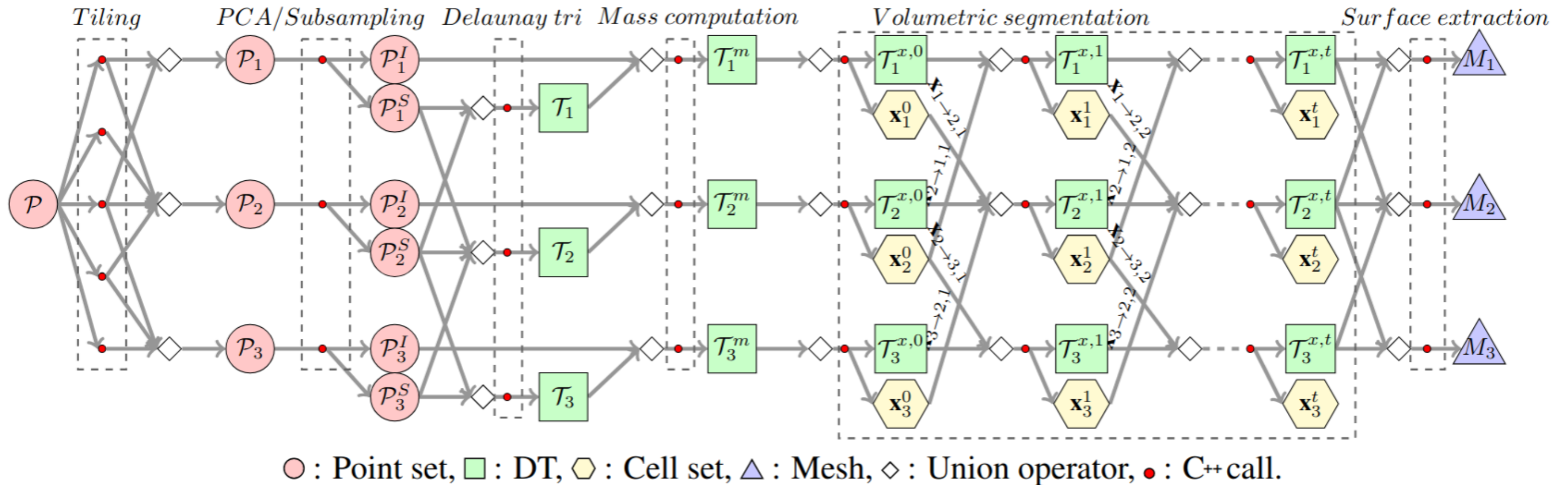
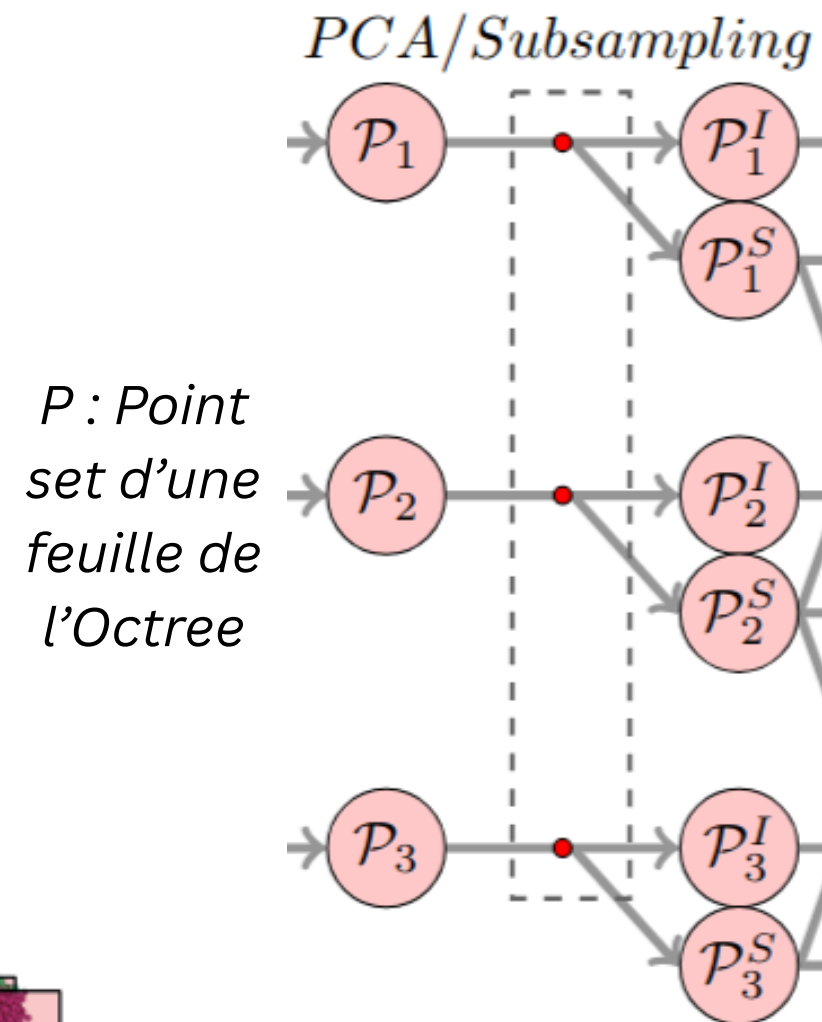
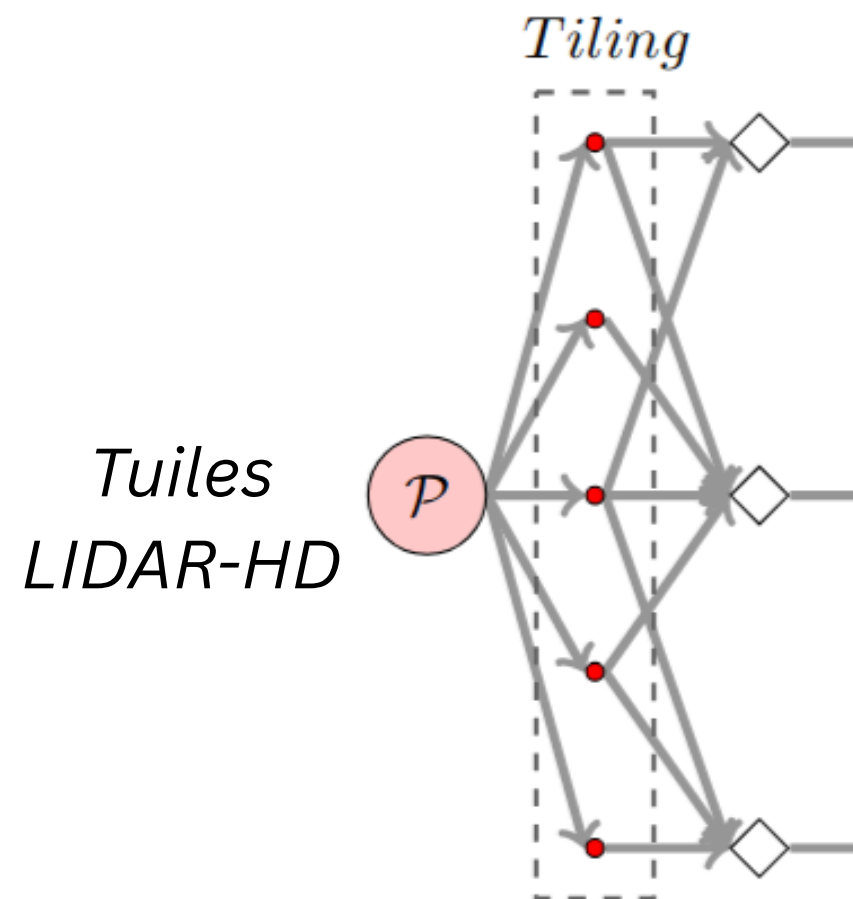


Schéma de l'algorithme de reconstruction de surface (ref 5)

Rappel

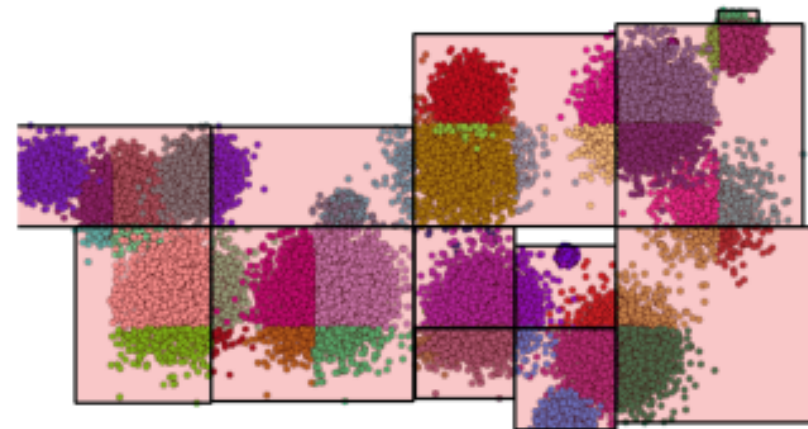
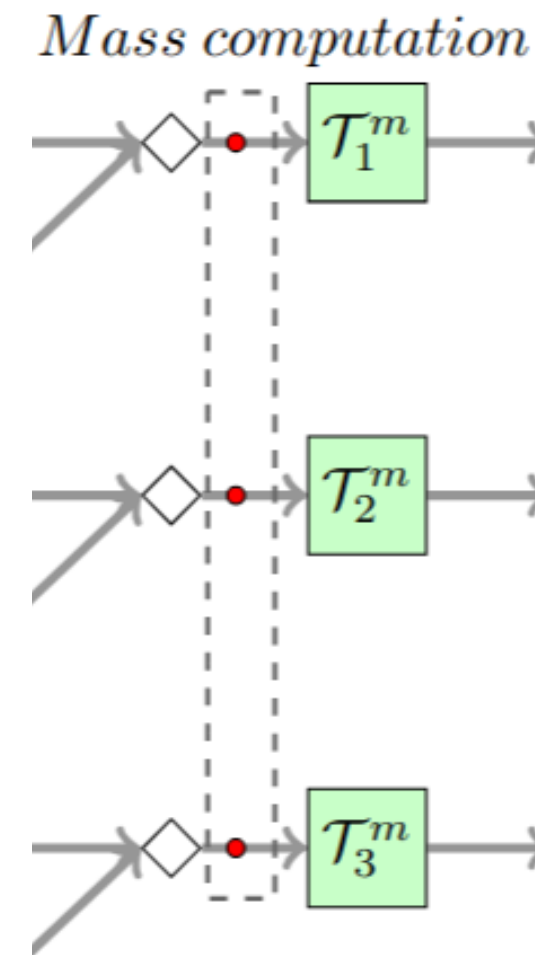
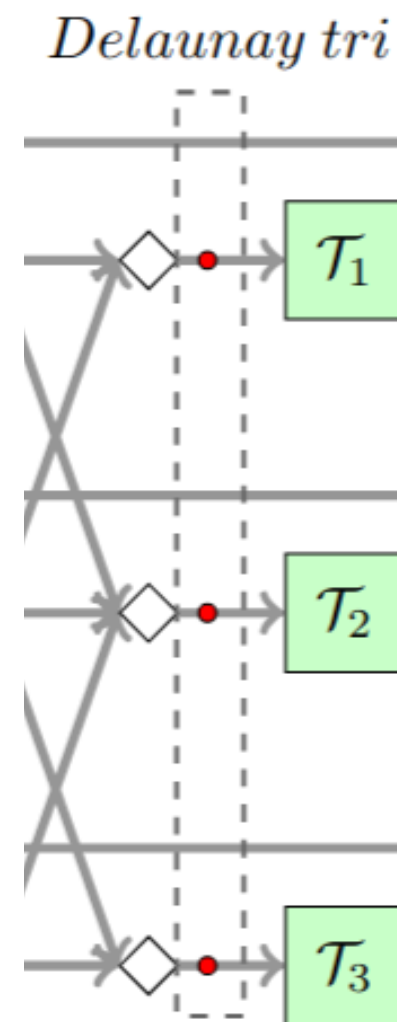
**T : Triangulation de
Delaunay locale**

T^m : Triangulation de
Delaunay locale labellisée

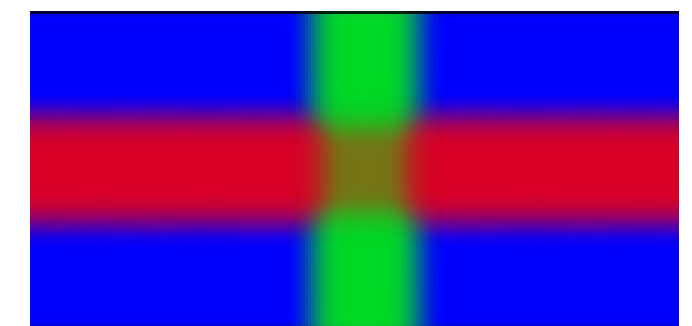
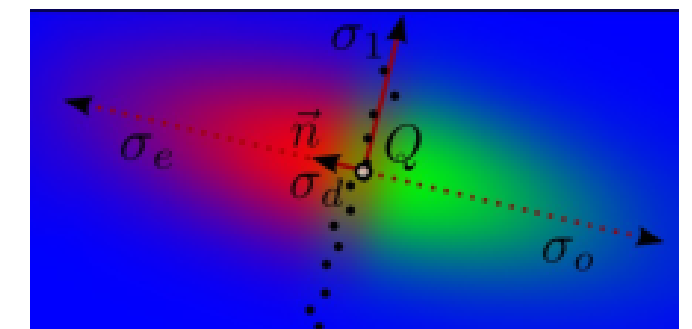
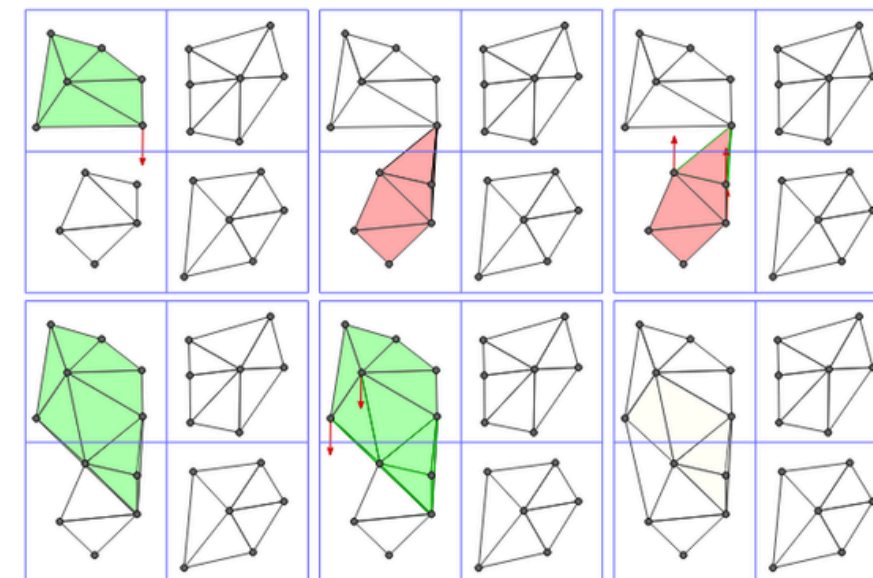
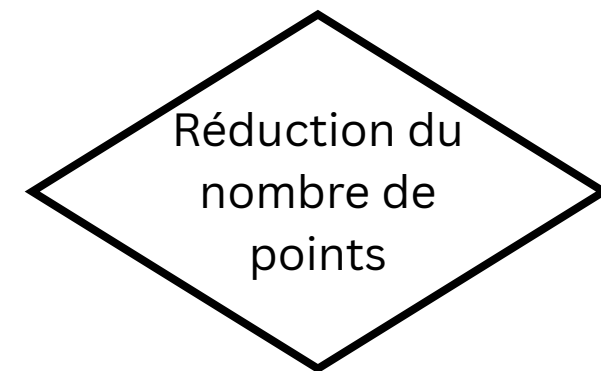


$\mathcal{P}^I$: Point set
d'une tuile
enrichie d'une
PCA

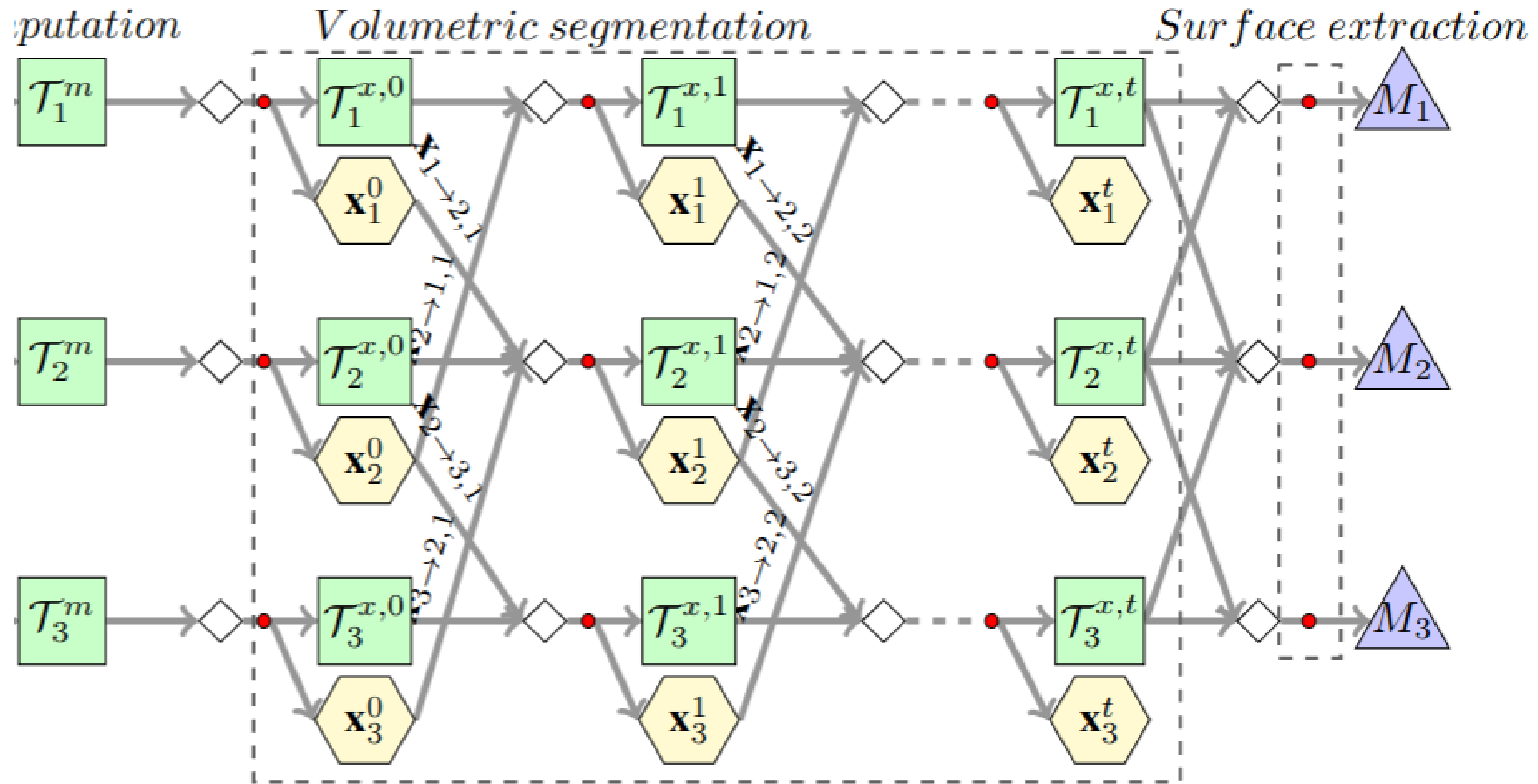
$\mathcal{P}^S$: Point set
simplifié
d'une tuile



Segmentation par tuile =>
Segmentation par nbr de points

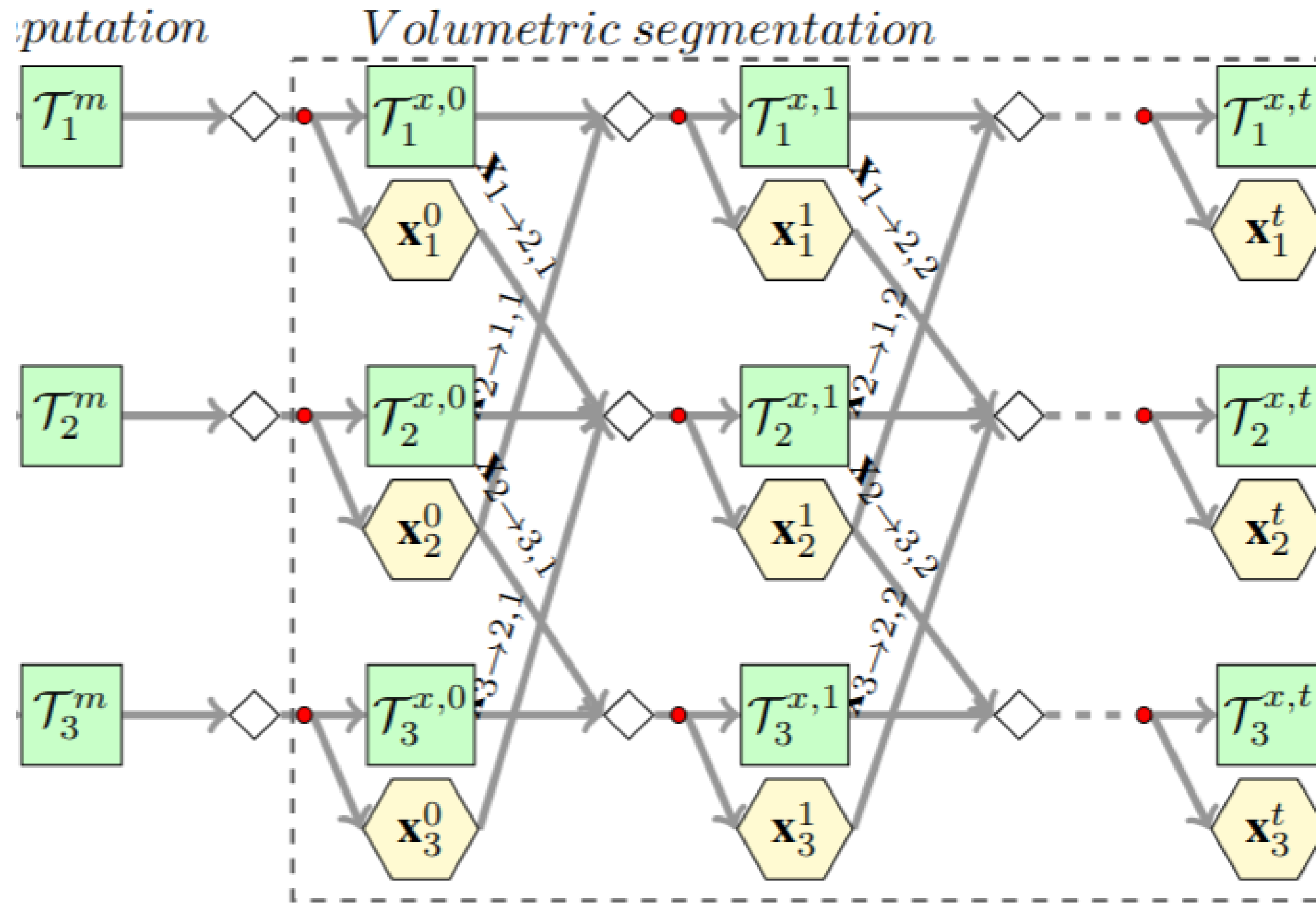


Segmentation volumétrique et extraction de surface



Segmentation volumétrique

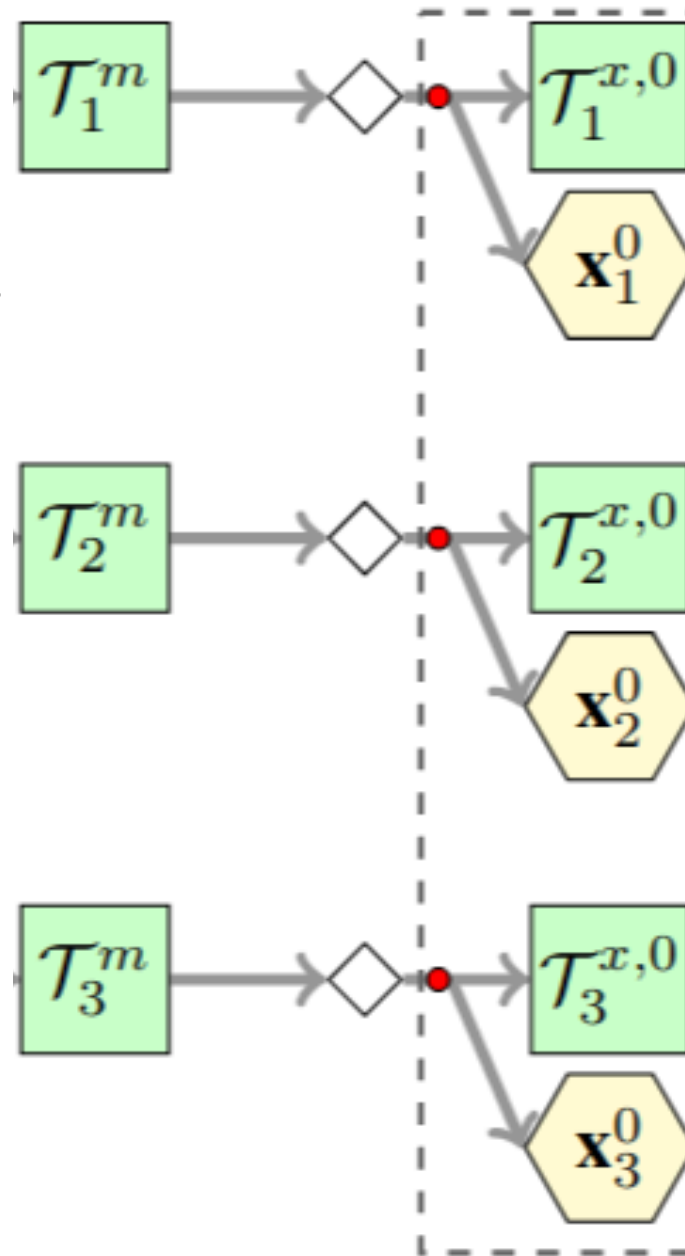
T^m : Triangulation
de Delaunay
locale avec
fonction de masse



$T^{x,t}$: Triangulation
de Delaunay
locale labellisé à
l'étape t

Segmentation volumétrique

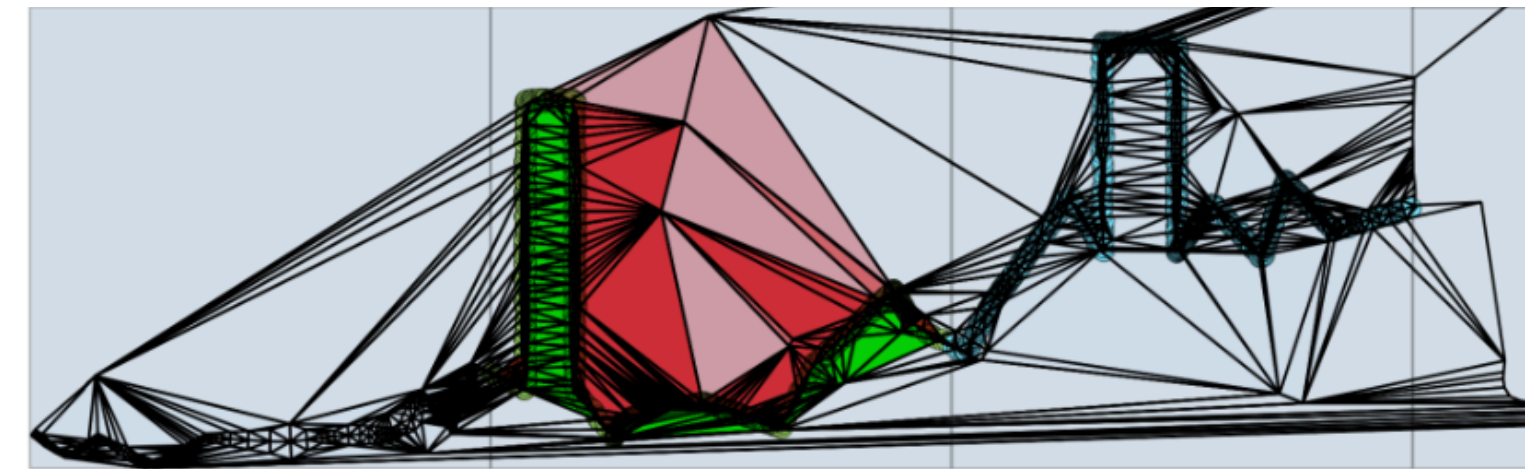
$\mathcal{T}^m$: Triangulation de Delaunay locale avec fonction de masse



$\mathcal{T}^{x,t}$: Triangulation de Delaunay locale labellisée à l'étape t

x^t : L'état des labels de chaque tétraèdre à l'étape t

=



Tétraèdres labellisés dans une tuile (ref 6)

Dans chaque tuile est attribué des labels temporaires à chaque tétraèdre

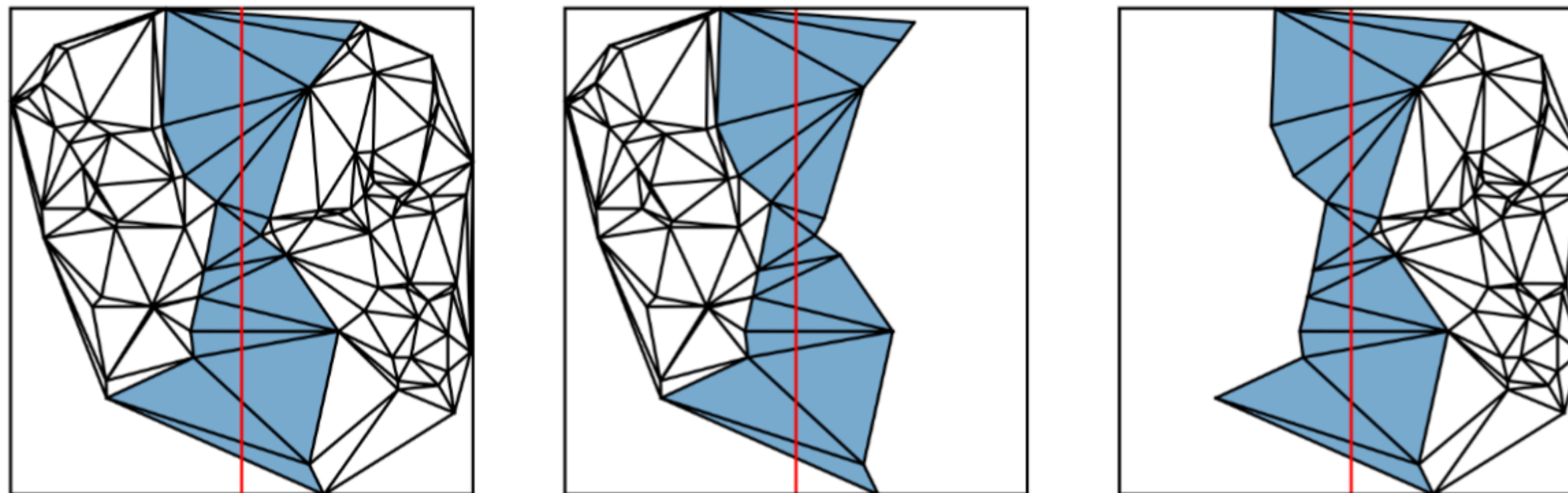
Minimisation d'énergie par graph-cut

- Énergie : fonction de coût globale (énergie faible = solution cohérente).
- Critères d'évaluation :
 - Fonction de masse (pénalité d'énergie si label contradictoire)
 - Cohérence spatiale (surface réel continue, relativement lisse et pas bruitée => 2 tétraèdres voisins ont tendance à être du même type)

Minimisation d'énergie = compromis optimal global (assez fidèle à la fonction de masse et régulier spatialement)

Processus d'itération

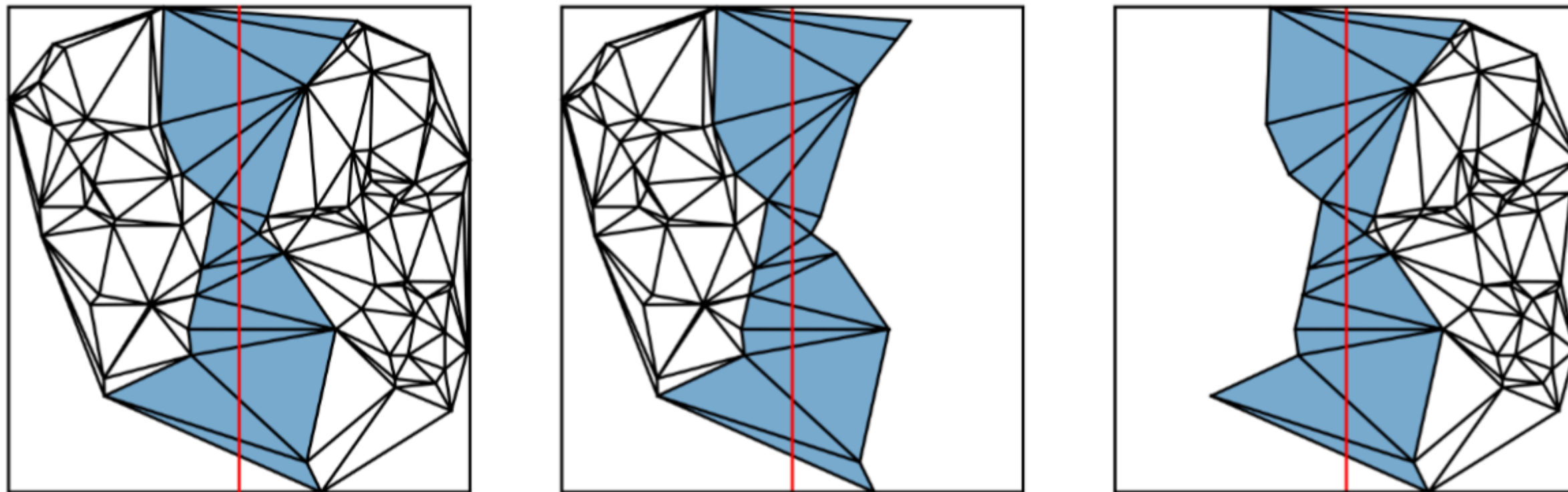
- Chaque tuile effectue un graph-cut local (algorithme convergent).
- Les labels des tétraèdres partagés entre tuiles peuvent être incohérents.
- On ajoute un terme qui permet l'évolution des labels lors des lancements de nouveaux graph-cut locaux. On répète jusqu'à convergence des labels des tétraèdres mixtes.



Cellules mixtes (ref 5)

Processus d'itération

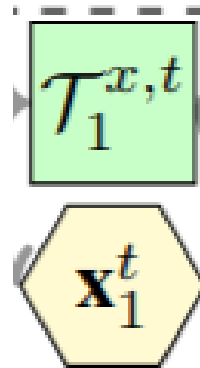
- Ce terme est un Lagrangien qui permet l'optimisation des contraintes. Il va forcer 2 tétraèdres au label contradictoire à choisir un label commun (descente de gradient).
- L'algorithme n'est pas assuré de converger. Un nombre fixe d'itérations maximum est donc choisis (30).



Cellules mixtes (ref 5)

Segmentation volumétrique

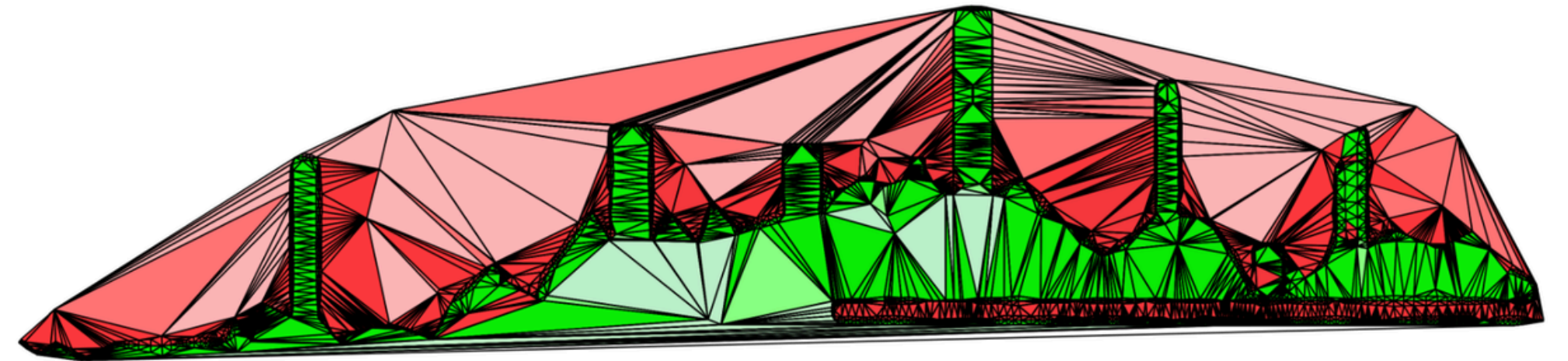
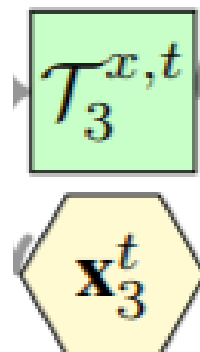
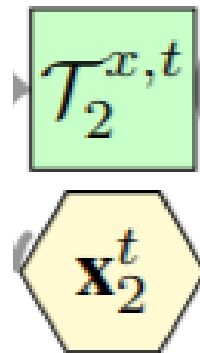
$\mathcal{T}^{x,t}$: Triangulation de
Delaunay locale
labellisée à l'étape t



État final

=

x^t : L'état des labels
de chaque tétraèdre
à l'étape t



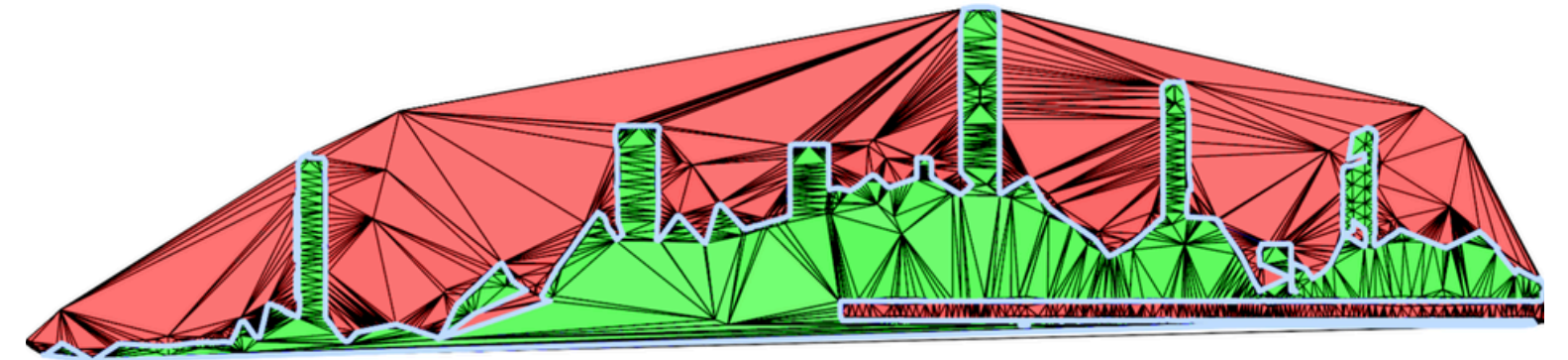
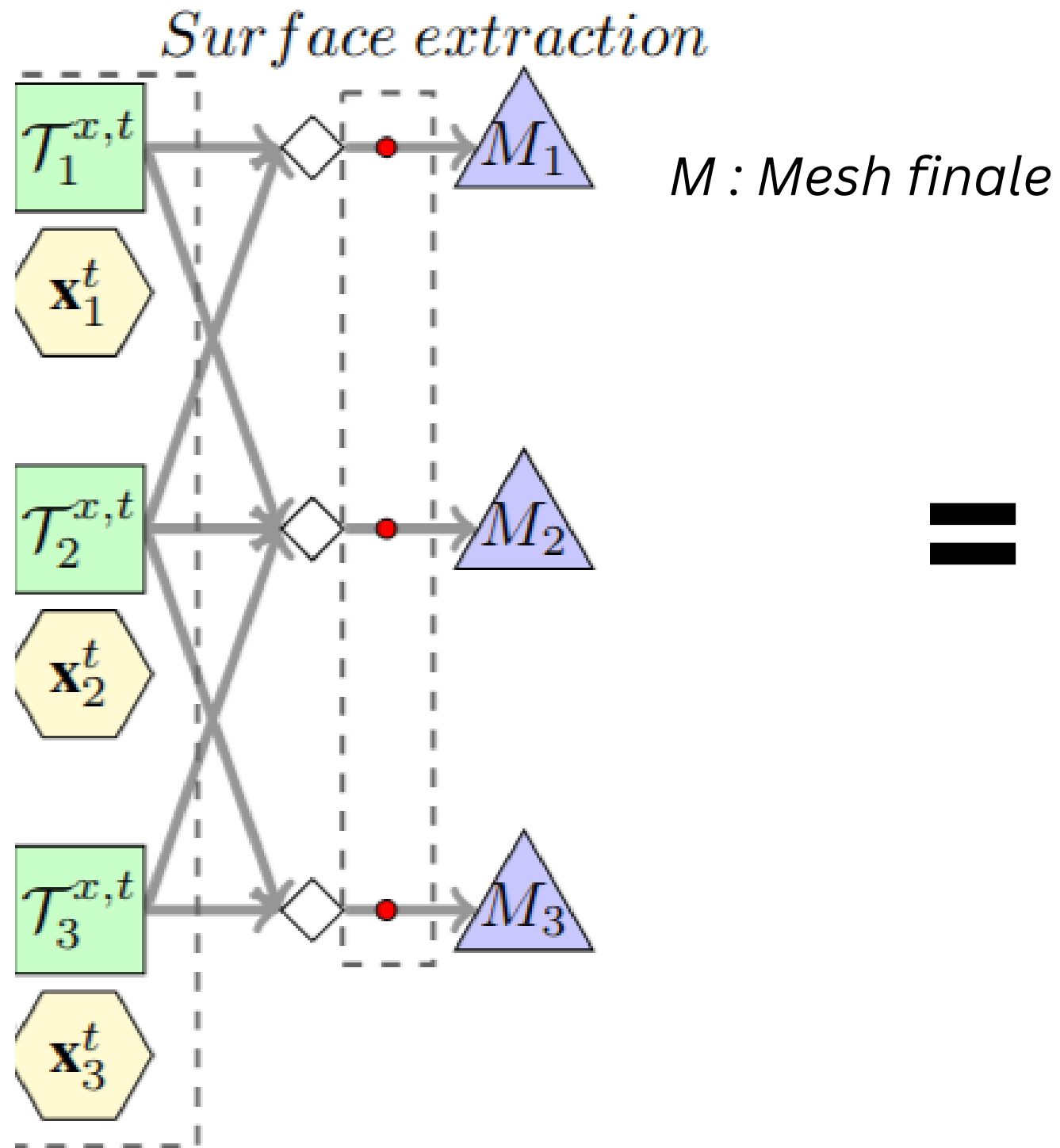
Labellisation des tétraèdres du modèle (ref 6)

**Les tétraèdres sont labellisés
“extérieurs” ou “intérieurs”.**

Extraction de surface

État final

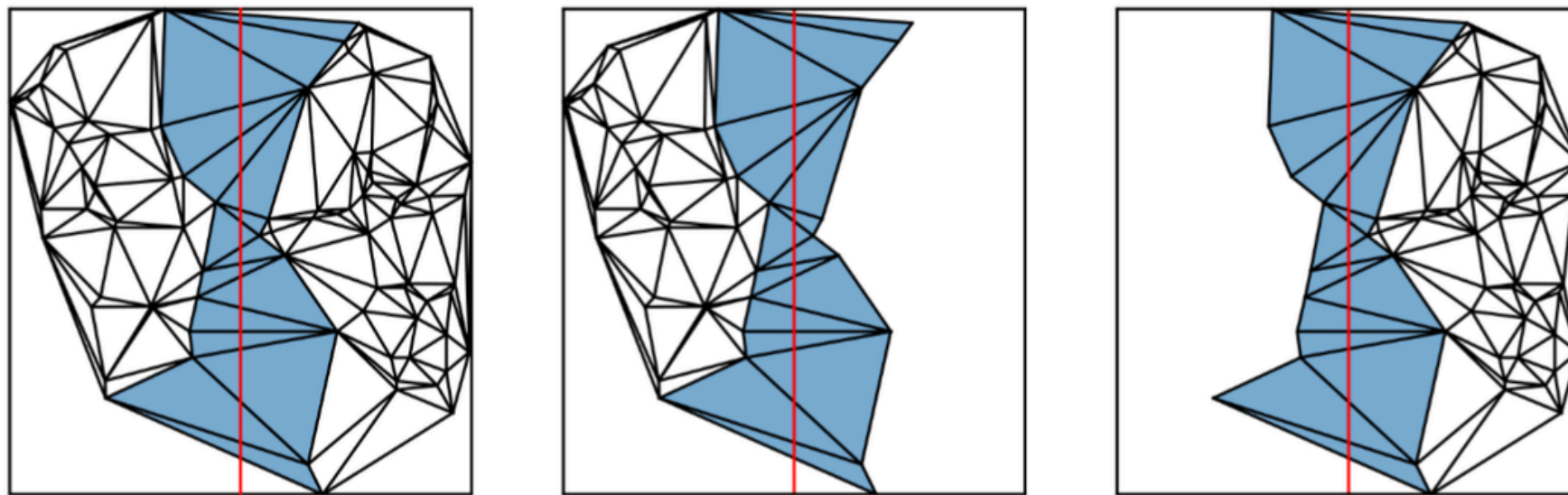
$\mathcal{T}^{x,t}$: Triangulation de
Delaunay locale
labellisé à l'étape t



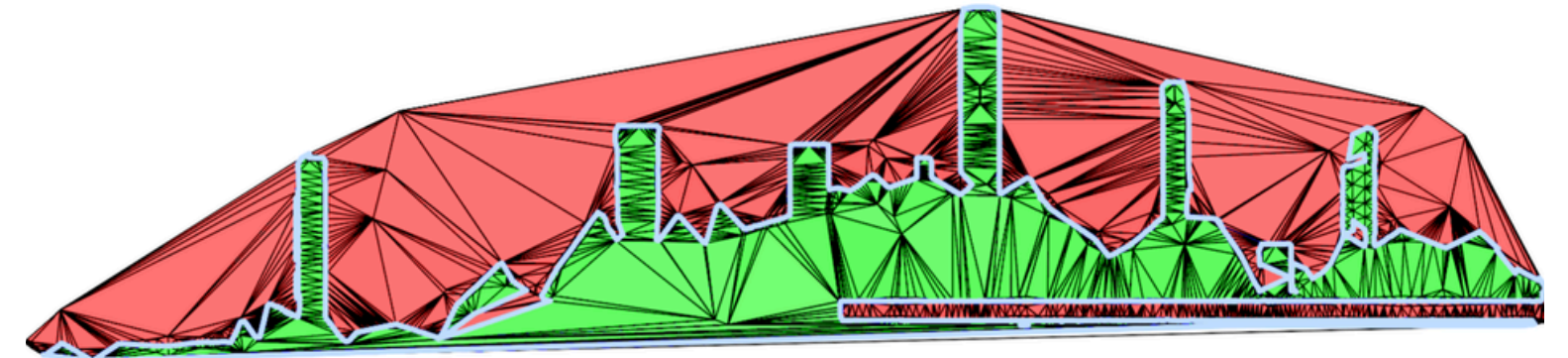
Surface (ligne blanche) du modèle (ref 6)

**La surface est composé des triangles situés
entre un tétraèdre plein et un tétraèdre vide.**

Extraction de surface



Cellules mixtes (ref 5)



Surface (ligne blanche) du modèle (ref 6)

Afin de garder l'unicité de la surface, la tuile ayant le plus petit id décide la sélection des triangles mixtes.

Améliorations

- **Changer la PCA pour du deep-learning**
- **Améliorer l'orientation des normales.**
- **Calibration des données**

Questions ?

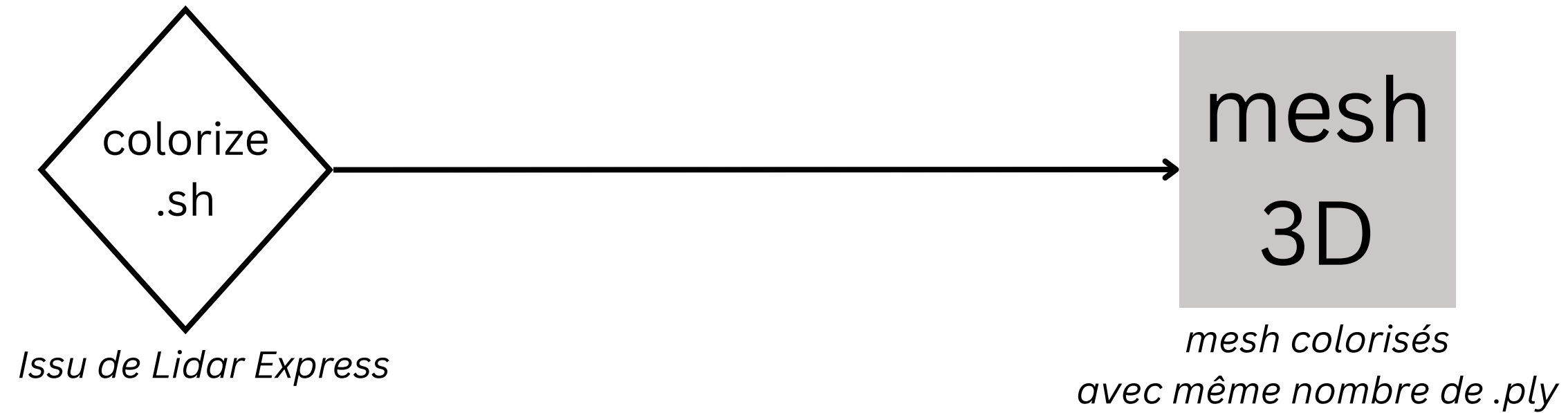
La migration de Spark vers Apache Airflow est-elle une bonne idée ?

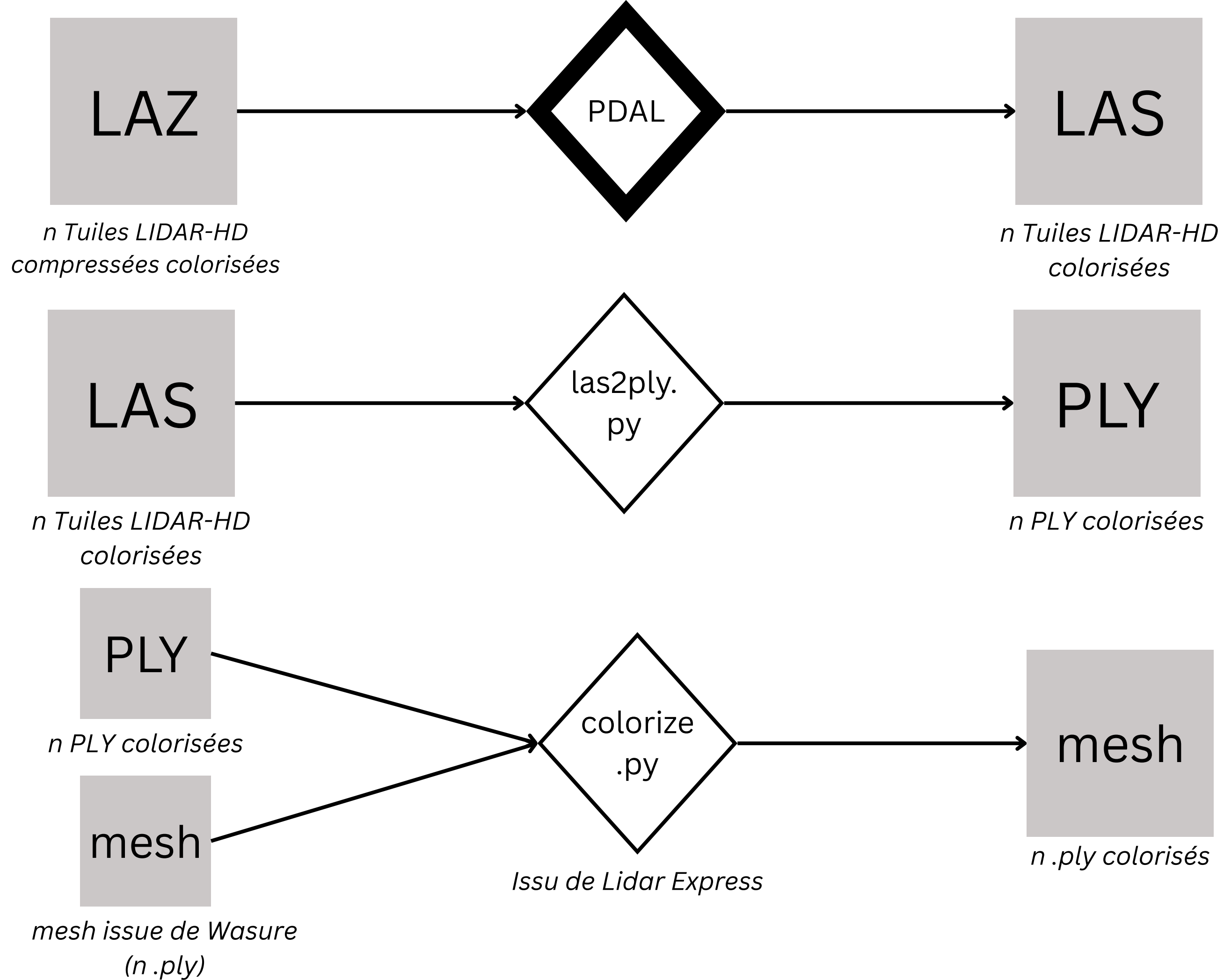
- **Spark permet de faire du calcul massif distribué à l'endroit où se situe la data (HDFS).**
- **Airflow fait du scheduling de process/job.**

Apache Airflow est-il capable de refaire du Spark aussi bien ?

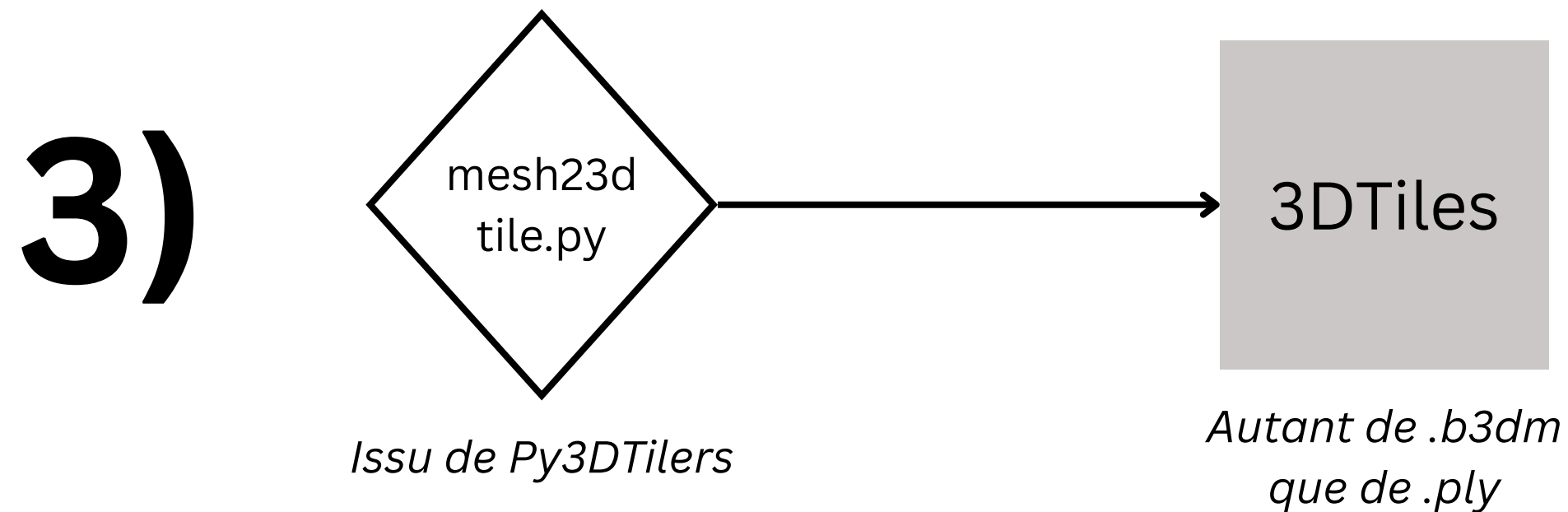
Colorize

2)



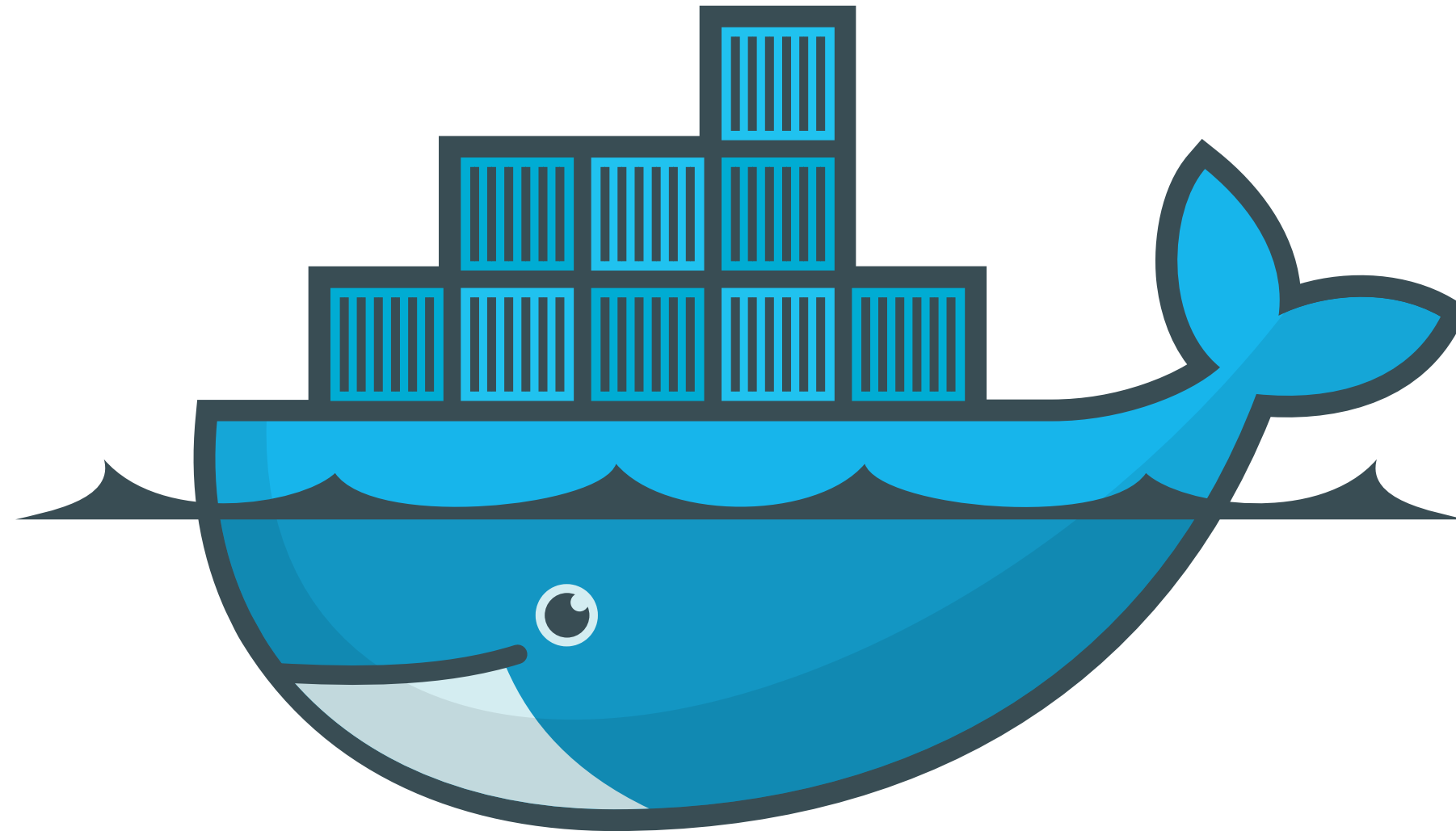


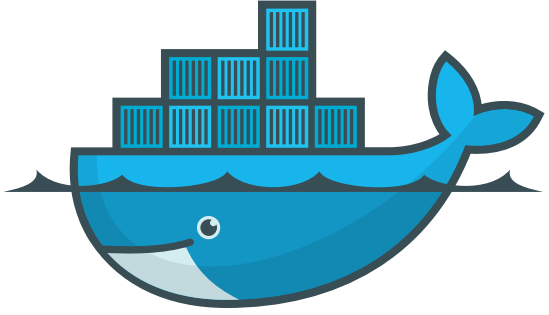
3DTiles



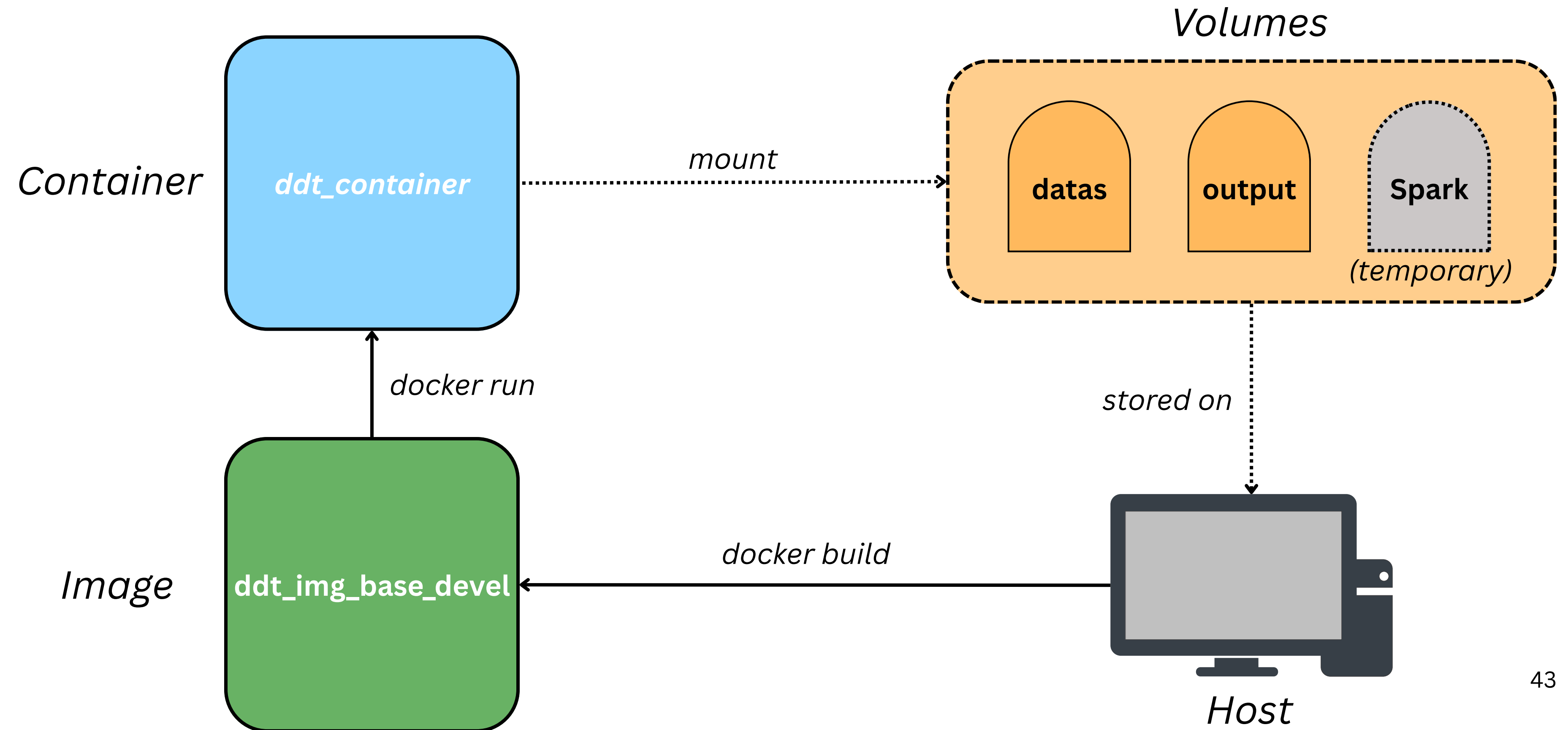
- Le nombre de fichier .b3dm (3DTiles) généré par mesh23tile.py correspond au nombre de fichier .ply en sortie de Wasure.
- La logique : pour chaque .ply => .b3dm
- Un système LODs (Level Of Details) est créé afin d'optimiser l'affichage

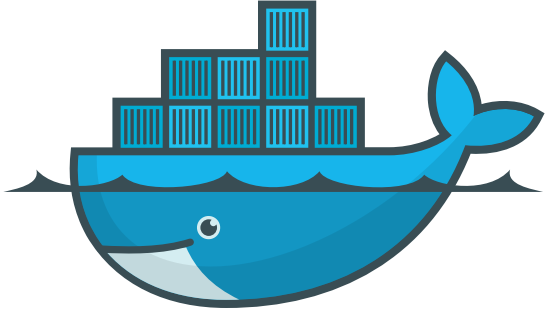
Docker





Docker





Docker

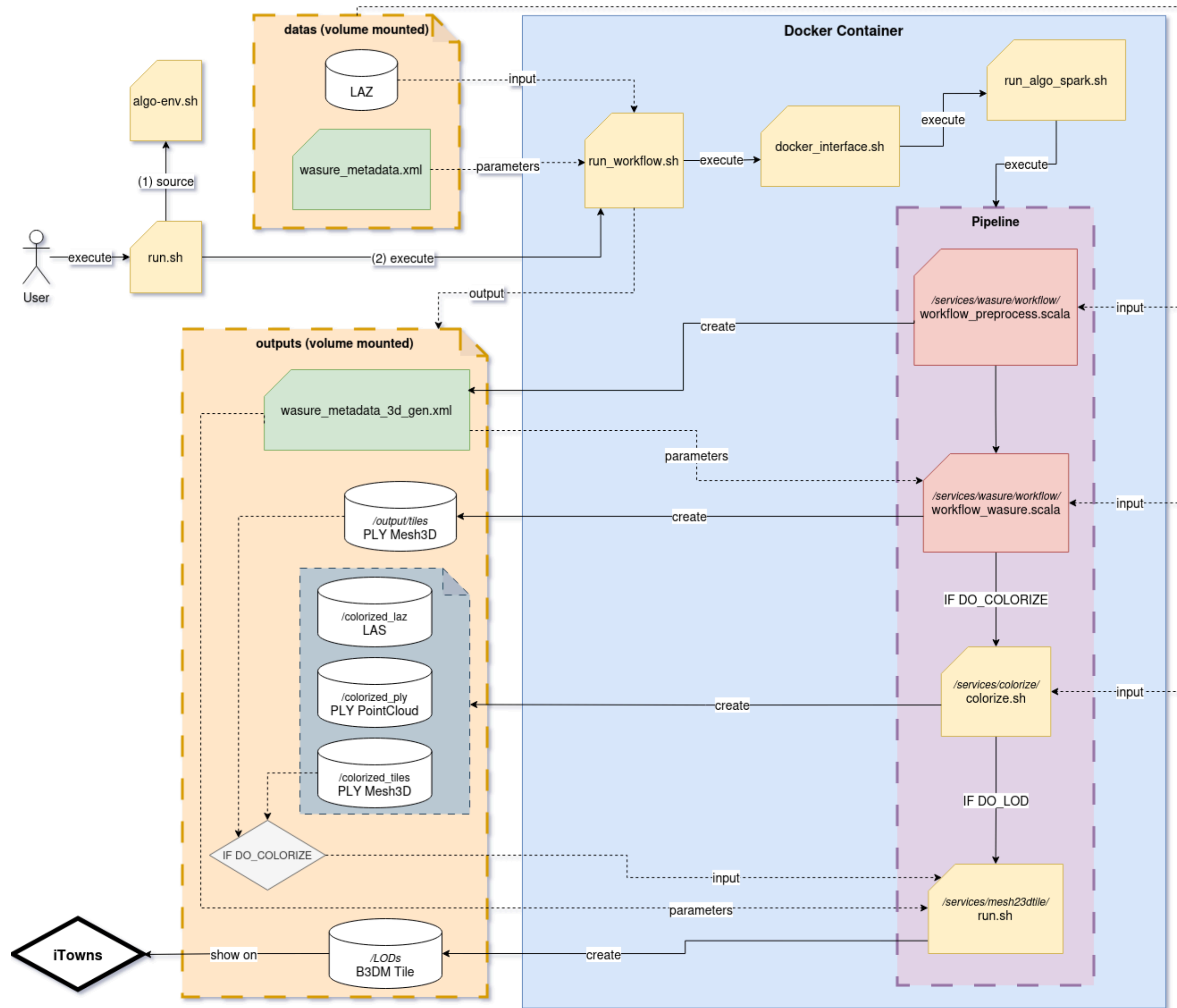
En résumé :

- **1 image**
- **1 conteneur**
- **3 volumes : datas (Tuiles LIDAR et metadata), output (les résultats) et un volume temporaire utilisé par Spark (volumineux)**

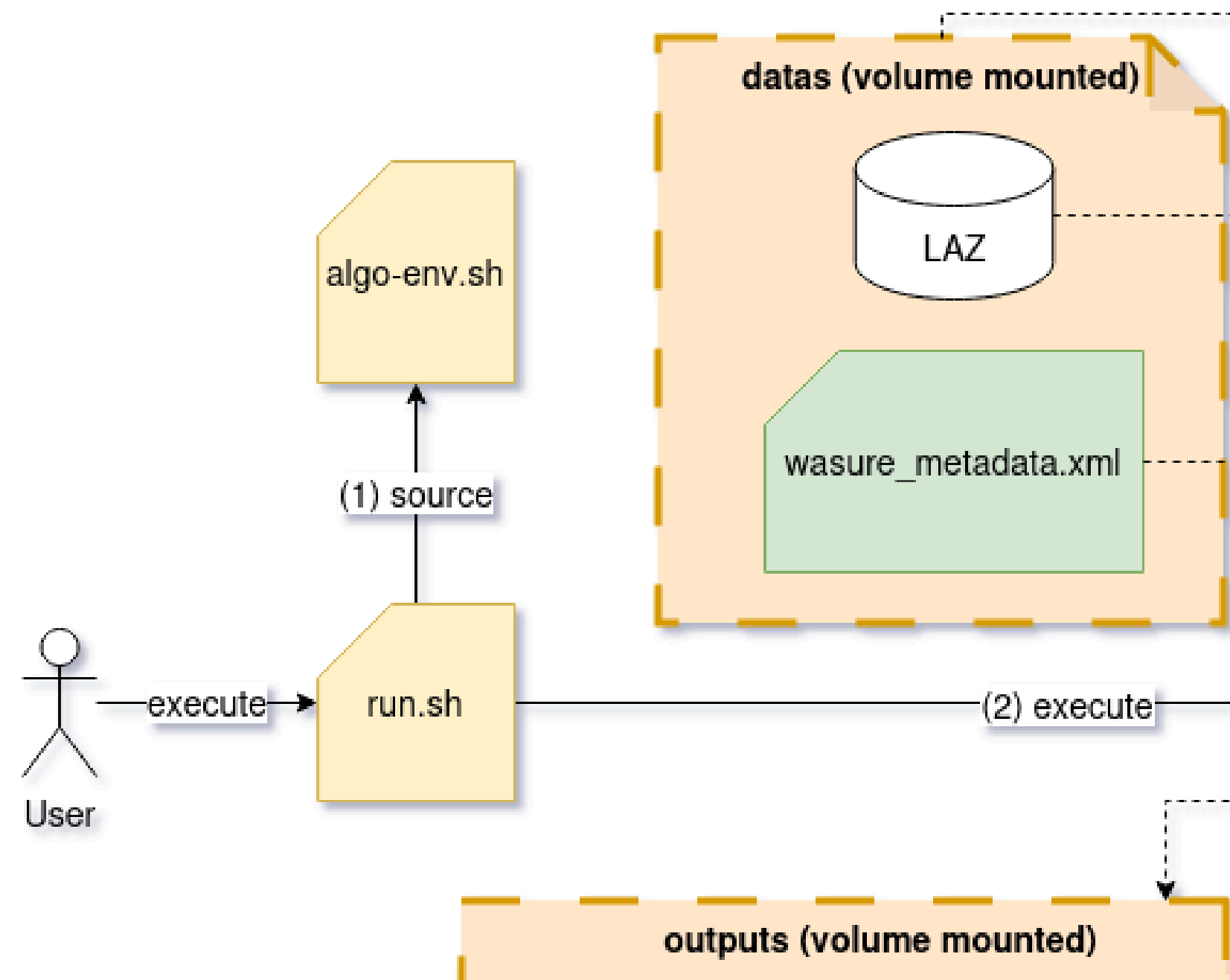
Références

1. 3D watertight mesh generation with uncertainties from ubiquitous data: <https://hal.science/hal-01882625v1>
2. 3D octree based watertight mesh generation from ubiquitous data: <https://hal.science/hal-01882668v1>
3. Distributed Delaunay Triangulations for Cloud Computing: <https://hal.science/hal-02535021v1>
4. Provably Consistent Distributed Delaunay Triangulation: <https://hal.science/hal-02551509v1>
5. Efficiently Distributed Watertight Surface Reconstruction: <https://hal.science/hal-03380593v1>
6. Diapo explicatif : <https://drive.google.com/file/d/1GxuTU6tWVP2iPH9FEo35dNO6s-Wnfhzi/view>
7. Vidéos des résultats : https://drive.google.com/drive/folders/1K04_BehhQ8plMG7vQ4GDrrTWDfrf5hAy
8. Repository GitHub: <https://github.com/lcaraffa/sparkling-wasure/tree/main?tab=readme-ov-file>

Le code



Étape 1

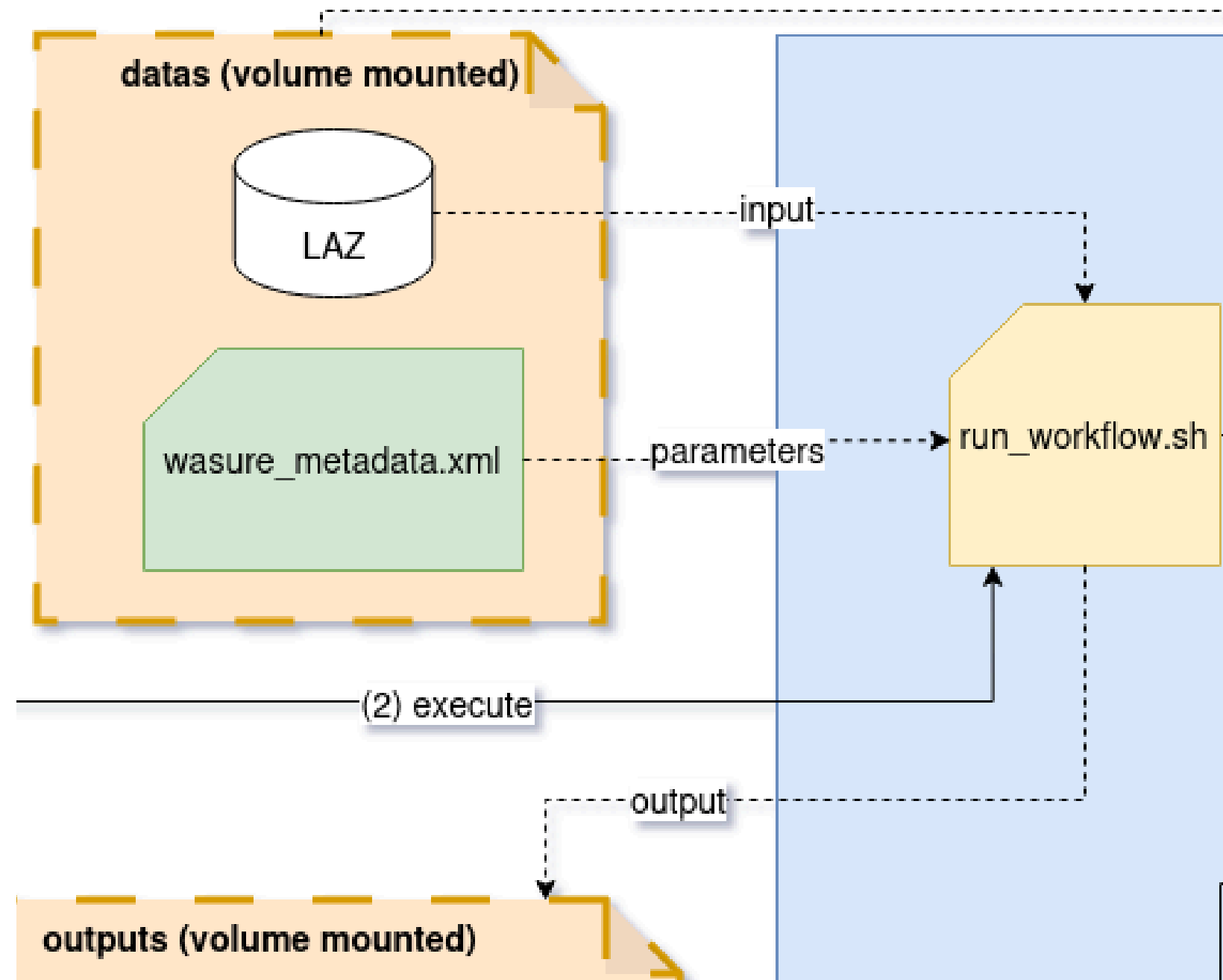


- `algo-env.sh` : export variables (Docker, Spark) et la fonction `run_cmd_container`
- `run.sh` : lance `run_cmd_container` avec inputs (tuile lidarhd), params (metadata xml), output (le chemin du dossier) et la commande à exécuter.

Améliorations

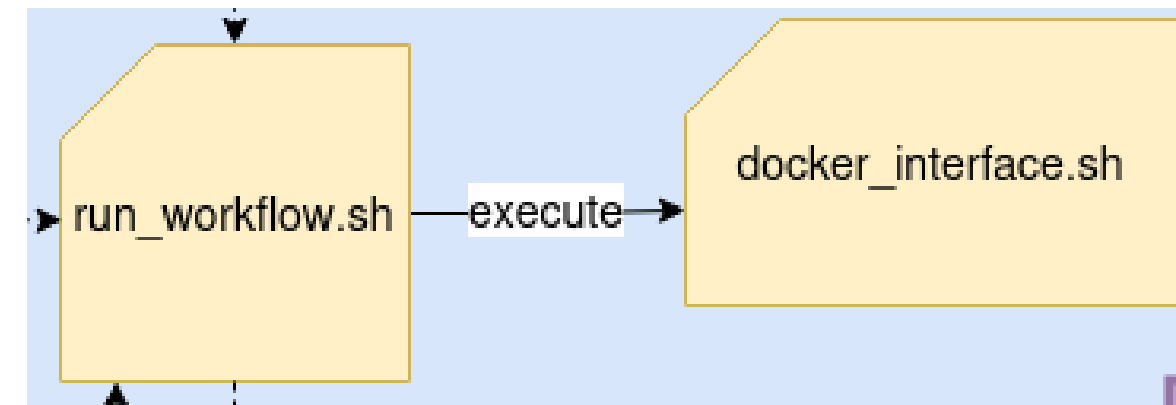
- **algo-env.sh : enlever la fonction, sh uniquement pour les variables**
- **run.sh : Pourquoi passer la commande en fonction alors que c'est toujours la même (exécution de run_workflow.sh) ?**

Étape 2



- **run_workflow.sh** : Script .sh s’occupant de l’orchestration de la pipeline.
- Exécute Wasure (les 2 .scala) puis colorize.sh et Mesh23dtile si les variables sont “TRUE”.

Étape 3



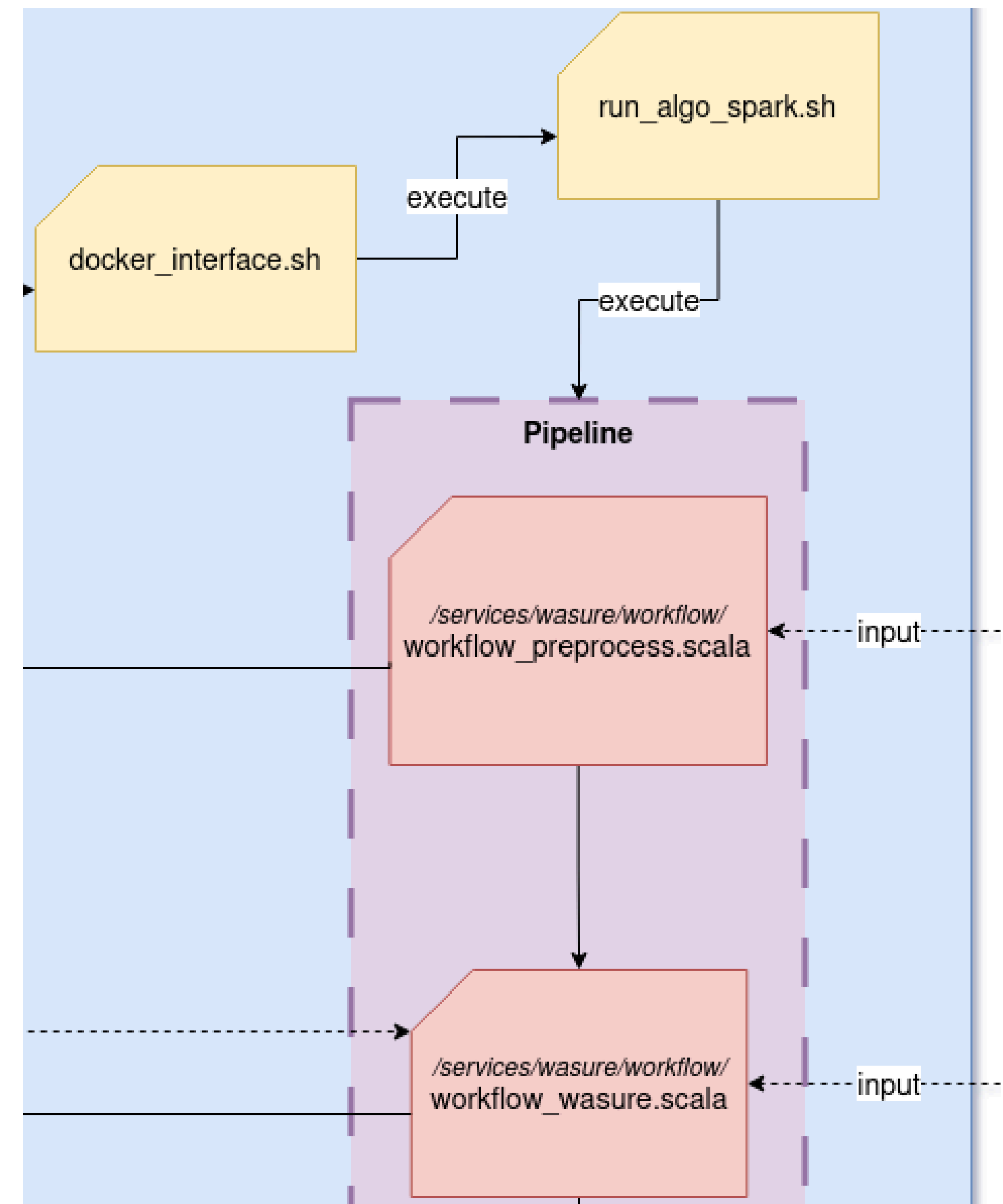
- **docker_interface.sh** : créer un container d'exécution s'il n'existe pas en appelant `run_bash_docker` (pour les tests), et lance l'appel à la pipeline spark.
- Ce script `.sh` contient également les fonctions de compilation (c++) et de build (services et spark) de l'image Docker.

Améliorations

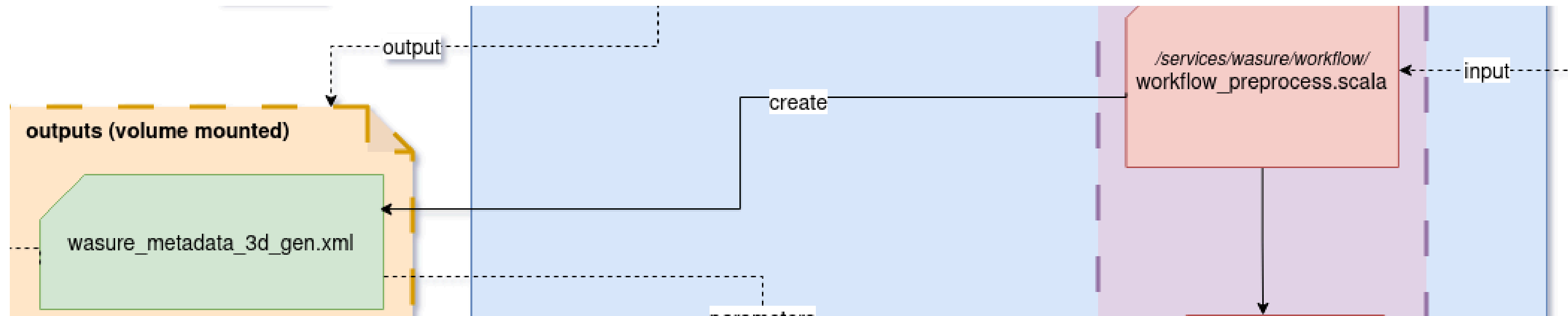
- Séparer la logique “Création d’image Docker” de l’aspect “Pipeline d’exécution”.
- Le contenu lié à la pipeline pourrait peut-être se situer dans un `.sh` déjà existant.

Étape 4

- **run_algo_spark.sh** : configure Spark et lance les process (workflow_preprocess.scala & workflow_wasure.scala)
- **Détails de la configuration :**
 - Environnement (mémoire, dossiers temporaires, ressources)
 - Démarrage cluster Spark dans le container Docker

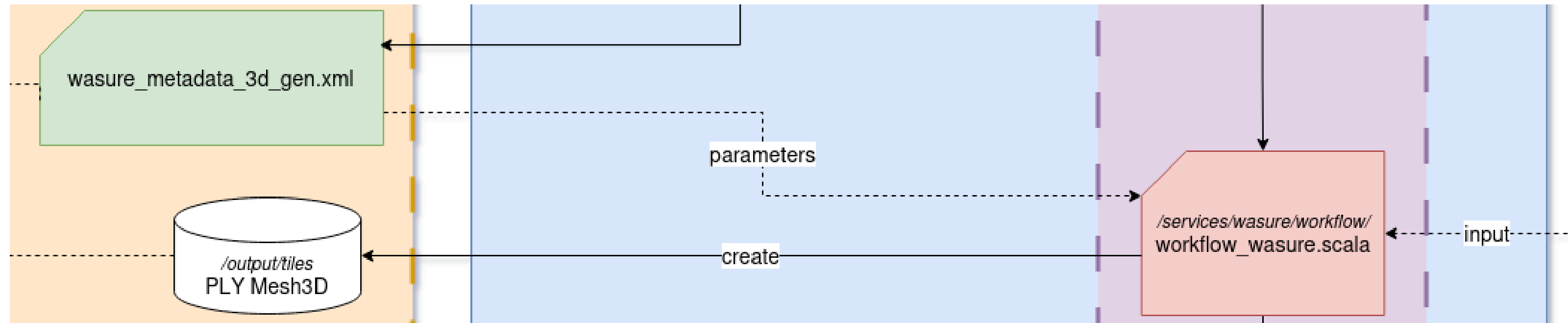


Étape 5



- **workflow_preprocess.scala** : Job Spark générant un fichier de paramètres (.xml) en parcourant l'ensemble des tuiles LIDAR-HD.
- **Exemple de paramètres** : bbox, profondeur de l'octree...

Étape 6

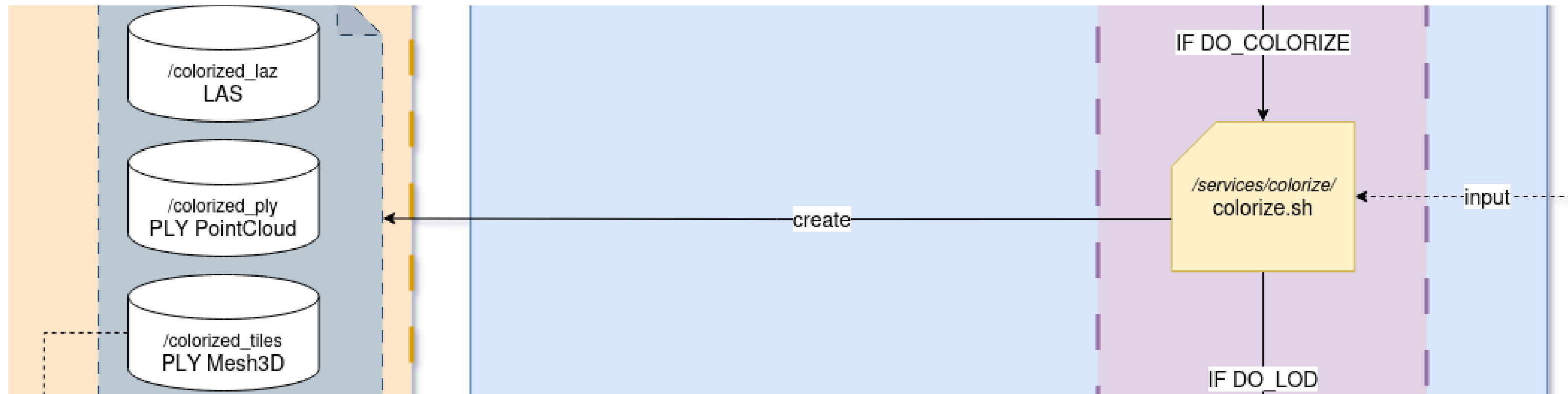


- **workflow_wasure.scala** : Job Spark exécutant l'algorithme de reconstruction de surface.
- Les 2 .scala n'exécutent pas directement les calculs géométriques, ils appellent des binaires C++ (créé à la compilation des fichiers C++ lors du build de l'image) qui sont orchestrés et distribués par Spark.

Améliorations

- **Enlever Spark du process : retirer les configurations (run_algo_spark.sh) et regrouper la logique d'exécution dans un seul exécutif.**
- **Supprimer les fichiers scala, mais conserver la pipeline d'exécutions des binaires C++. Appliquer cette dernière dans Apache Airflow.**

Étape 7

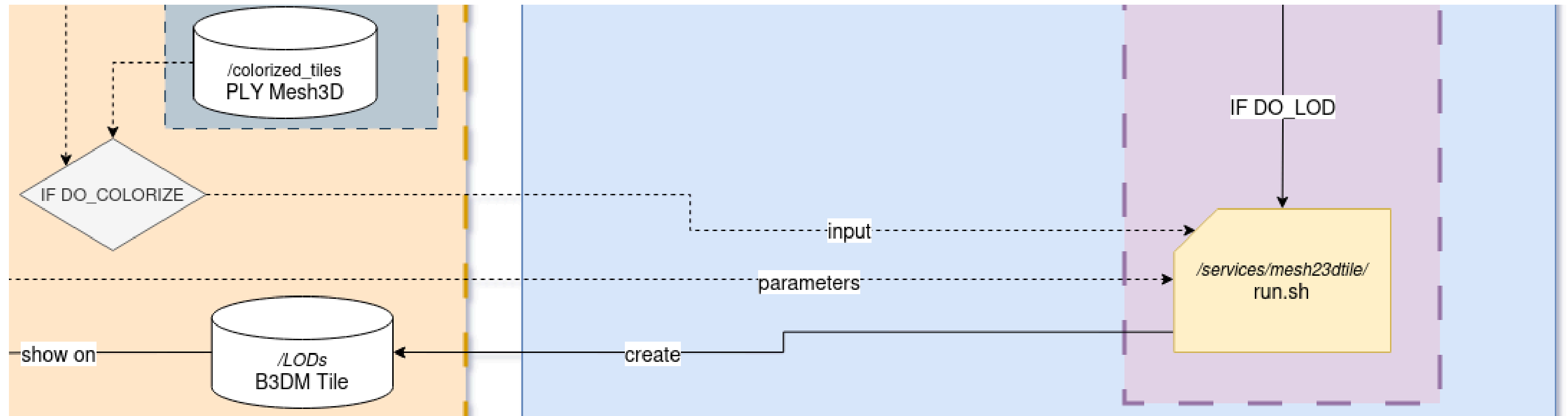


- **colorize.sh** : Génère des nuages de points colorés depuis les tuiles LIDAR-HD, les converties en .ply, puis applique les couleurs aux Mesh3D sortie de Wasure.
- Cette dernière étape se fait par application des couleurs sur les sommets des mesh3d .

Améliorations

- **Enlever temporairement le rendu colorimétrique.**
- **Réfléchir à une logique d'intégration du process, définir si d'autres data sont utiles à conserver.**

Étape 8

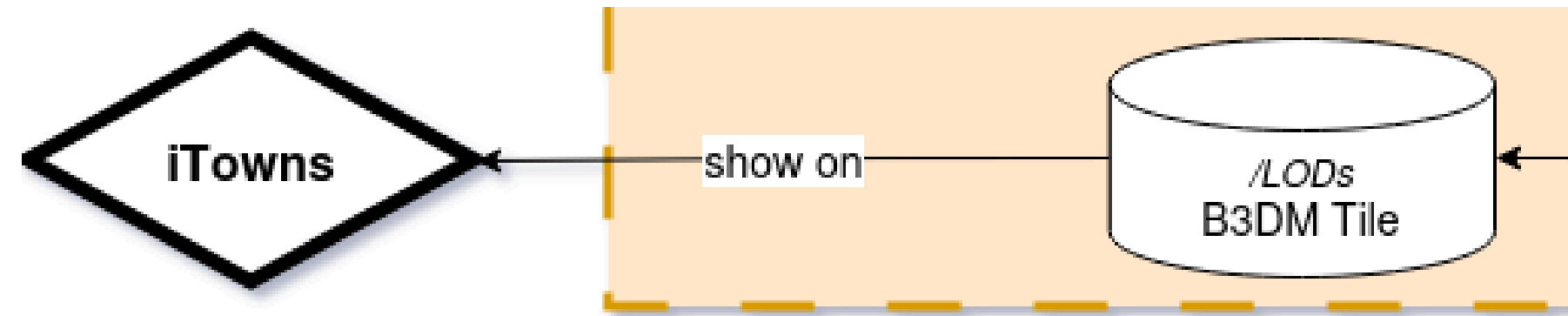


- **run.sh** : Utilise le service “mesh23tile” afin de transformer les mesh3D en 3DTiles (.b3dm). Les fichiers .ply sont d’abord transformés en .obj (géométrie locale) puis en fichier .b3dm.
- Une hiérarchisation LOD avec un fichier `tileset.json` est produit. Les traitements sont parallélisés via `job process` et non pas par Spark.
- Les 3DTiles sont géoréférencés en EPSG:4978 (ECEF).

Améliorations

- **Est-ce que la logique de découpage des données issue de Wasure est idéale pour l'affichage des données ?**
- **Faut-il changer le système de parallélisation (Job process) pour Apache Airflow ? Est-il possible de conserver ce système de parallélisation ?**

Étape 9



- Étape en dehors de Wasure : le dossier de LODs peut être chargé dans iTowns (le fichier tileset.json gère tout).
- Le RSC EPSG:4978 (ECEF) permet l’affichage sur globe (GlobeView).

Problèmes

- Les fichiers .b3dm apparaissent en dessous de l'ellipsoïde du globe iTowns.
- Il est très compliqué de les faire translater par rapport au “z de la surface”, à cause de la projection géocentrique.